



AETN 4301 Capstone Project II
Department of Telecommunications and Network Engineering
College of Engineering Technology
University of Doha for Science and Technology

BACHELOR OF SCIENCE IN APPLIED TELECOMMUNICATIONS AND NETWORK ENGINEERING

**Design and Implementation of a Large Language Model-Driven Intrusion
Detection System for Edge-enabled Smart Cities**

Students:

- Ali Suhail 60106420
- Khaled Rahman 60104156
- Abdallahi Mahmoud 60300336

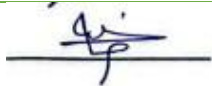
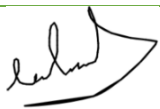
Supervisor: Dr Dana Haj Hussein
Course instructor: Dr Salman Saadat

Date: 13/02/2026

DECLARATION STATEMENT

We, undersigned students, hereby declare that this project report and work described in this report is entirely our own work and has not been copied from any other source. Any material that has been used from other sources has been cited and acknowledged in proper style.

We are totally aware that any copying or improper citation of references/sources used in this report will be considered plagiarism, which is a clear violation of Academic Dishonesty Policy at the University of Doha for Science and Technology (UDST).

	Student Name	Student ID	Signature	Date
1	Ali Suhail	60106420		15/11/2025
2	Khaled Rahman	60104156		15/11/2025
3	Abdallahi Mahmoud	60300336		15/11/2025

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to Dr. Dana Haj Hussein, our Capstone supervisor, for her continuous guidance, valuable feedback, and encouragement throughout this project. We also thank Dr. Salman Saadat for his academic support and insightful advice.

Our appreciation extends to the School of Engineering Technology at the University of Doha for Science and Technology (UDST) for providing the tools and facilities necessary to complete this work. Finally, we thank all those who contributed to our learning and success during this project.

Abstract

The rise of IoT devices and edge computing frameworks within smart cities have broadened the IoT cybersecurity attack surface. This project presents and defends the concept of an Intrusion Detection System powered by Large Language Models (LLMs) that interprets, summarizes, and categorizes edge smart city environments' IDS alerts. Leveraging OpenAI's GPT-4 and LangChain, the proposed system interprets raw IDS logs and outputs understandable texts relating to low-tech and high-tech security concepts, evaluates their severity, proposes mitigation strategies, and describes the security context. This solution was developed within a Kubernetes-based edge deployment for scalability, low latency, and resilience. Initial prototype tests showed that the LLM was capable of translating extremely complicated alerts into succinct, sensible, and pertinent summaries, substantially alleviating the mental load of the operators. The project will promote the comprehension of current threats, facilitate smart cities' security systems, and allow the environments to be more user-friendly, and improve the effectiveness of the systems.

Table of Contents

1	Chapter 1: Introduction	0
1.1	Background & Motivation: (Importance of the Project, Context and Stakeholders) .	1
1.2	Problem Statement.....	2
1.3	Objectives	4
1.4	Expected Impact	7
2	Chapter 2: Literature Review	8
2.1	Review of Related Work: (Existing Solutions, Products, or Research, Analysis of Strengths and Gaps)	10
	Summary of literature weaknesses:	11
	Gap this project addresses:	12
2.2	Technical Background: (Key theories, principles, and technologies involved)	12
3	Chapter 3: Project Requirements and Specifications	13
3.1	Customer / Stakeholder Needs	13
	Primary Stakeholders:	13
	High-Level Needs Summary:.....	14
3.2	Requirements	15
3.2.1	User Experience Considerations	15
	User Experience Requirements:	15
3.2.2	Functional Requirements	15
	Core Functional Requirements:	16
3.2.3	Non-Functional Requirements	16
	Performance Requirements:	17
	Reliability Requirements:.....	17
	Scalability Requirements:.....	18
	Security Requirements:	18

Usability Requirements:	18
Maintainability Requirements:	18
3.3 Design Constraints (Cost, Power, Size, Environmental Factors)	19
Technical Constraints:	19
Environmental Constraints:	19
Economic & Cost Constraints:	20
Regulatory & Compliance Constraints:	20
Operational Constraints:	21
3.4 Design Considerations of public health, safety, welfare, global, cultural, social, environmental, and economic factors	22
Public Health:	22
Safety:	22
Welfare	22
Global Factors:	22
Cultural Factors:	23
Social Impact:	23
Environmental:	23
Economic	23
3.5 Standards and Regulations (IEEE, IEC, etc.)	23
3.6 Risk Assessment and Mitigation	25
3.7 Sustainable Development Goals (SDGs) Mapping	27
3.8 Economic evaluation, Business Model Canvas (BMC)	28
4 Chapter 4: Methodology & Design Approach	30
4.1 Introduction to Methodology	30
4.2 System Architecture (Block Diagrams, Functional Flow)	33
4.3 Concept Generation and Selection	35

4.3.1	Morphological Matrix	36
4.3.2	Evaluation of Alternative Concepts / Trade Studies	36
4.3.3	Justification of Chosen Concept.....	37
4.4	Modeling and Calculations	39
	Alert Throughput Modeling:	39
	LLM Response Time Modeling	40
	End-to-End Alert Latency	41
	Storage Modeling	42
	API Cost Modeling (GPT-4).....	43
4.5	Hardware Design Approach (Circuit Design, Components, PCB Layout)	43
	Edge Hardware Overview	43
	Circuit / Network Layout	45
	Hardware Security Considerations:	45
4.6	Software Design Approach (if applicable) (programming languages, algorithms, flowcharts).....	46
	Technology Stack:.....	46
	Data Processing Algorithm (Falco → LLM → Action)	47
	LLM Prompt Template	47
	Flowchart (Backend Processing).....	48
4.7	List of Required Components (Hardware & Software).....	49
	Hardware Components.....	49
	Software Components	49
4.8	Project Planning (Work Packages, Timeline & Milestones)	50
	Work Breakdown Structure (WBS)	50
	Gantt Chart (Capstone I)	51
4.9	Project Budget	51

4.10	Project Management (Team Structure, Roles, and Responsibilities).....	53
4.11	Expected Deliverables: (What the project will produce at the end of Capstone 1 and Capstone 2)	54
	Capstone I Deliverables:	54
	Capstone II Deliverables:	55
5	Chapter 5: Implementation.....	59
5.1	Hardware Implementation.....	59
5.2	Software Development.....	61
5.3	Prototype Development.....	61
5.4	Economic evaluation.....	63
6	Testing and Results.....	64
6.1	Testing Methodology	64
6.2	Experimental Setup.....	64
6.3	Results.....	65
6.4	Performance Evaluation.....	67
6.5	Limitations and Challenges.....	67
6.6	Improvements and Optimization.....	67
7	Conclusions and Future Work	68
7.1	Project Summary.....	68
7.2	Contributions and Achievements.....	69
7.3	Future Enhancements.....	69
8	References	70
9	Appendices.....	74

List of Figures

Figure 1: Overall System Architecture	14
Figure 2: LLM-Enhanced Intrusion Detection Workflow	35
Figure 3: Circuit / Network Layout	43
Figure 4: LLM Processing Pipeline	46
Figure 5: Flowchart (Backend Processing)	47
Figure 6: Work Breakdown Structure (WBS).....	49
Figure 7: Gantt Chart (Capstone I)	50
Figure 8: Gantt Chart (Capstone II)	50
Figure 9: Project Management (Team Structure)	53
Figure 10: Kubernetes cluster	55
Figure 11: IoT-IDS Integration Architecture	62
Figure 12: Cluster Scalability Under Alert Load	66
Figure 13: Key Contributions Map	68

List of Tables

Table 1: System Objectives and Evaluation Criteria	5
Table 2: Risk Assessment and Mitigation Strategies	25
Table 3: Relevant SDGs and Project Contribution	28
Table 4: Business Model Canvas (BMC)	29
Table 5: Morphological Matrix for System Design	36
Table 6: Typical values	42
Table 7: Selected Edge Node Specifications.....	44
Table 8: Hardware Components.....	49
Table 9: Software Components	49
Table 10: Capital Expenditure (CAPEX).....	51
Table 11: Operational Expenditure (OPEX)	52
Table 12: Development Costs	52
Table 13: Roles	53
Table 14: Roles	53
Table 15: Roles	53

Table 16: Roles	53
Table 17: Roles	53

Glossary of Terms / Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
BMC	Business Model Canvas
CAPEX	Capital Expenditure
CPU	Central Processing Unit
CNCF	Cloud Native Computing Foundation
DNS	Domain Name System
ECC	Error-Correcting Code
ELK	Elasticsearch, Logstash, Kibana
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IDS	Intrusion Detection System
IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
K8s	Kubernetes
LLM	Large Language Model
NIC	Network Interface Card
NIST	National Institute of Standards and Technology
OPEX	Operational Expenditure
PCB	Printed Circuit Board

RBAC	Role-Based Access Control
RTT	Round-Trip Time
SDGs	Sustainable Development Goals
SIEM	Security Information and Event Management
SOC	Security Operations Center
SQL	Structured Query Language
SSD	Solid-State Drive
TCP/IP	Transmission Control Protocol/Internet Protocol
TPM	Trusted Platform Module
UDST	University of Doha for Science and Technology
UPS	Uninterruptible Power Supply
VPN	Virtual Private Network
WBS	Work Breakdown Structure

1 Chapter 1: Introduction

Before delving into the specifics of this project, it is important to understand why integrating Large Language Models (LLMs) with intrusion detection systems is particularly valuable for smart city security. As this domain has become highly integrated with IoT and edge computing technologies, these have emerged as core elements of modern urban infrastructure [1]. Consequently, cybersecurity in these distributed systems has become a primary concern for city administrators and security professionals [2]. The major challenge is the lack of automated security tools that provide precise, actionable intelligence in real-time [3]. This project addresses several urban technology security issues through its aims and objectives, with the core goal of minimizing response latency in urban cybersecurity operations. These aspects will be discussed in detail in the following sections.

The proposed system architecture, illustrated in Figure 1, demonstrates a layered approach to intelligent threat detection and automated response. At the infrastructure layer, the system operates on an Azure Virtual Machine (Standard B2ms: 2 vCPU, 4GB RAM) hosting a single-node K3s Kubernetes cluster on Ubuntu Server 22.04 LTS, connected via Azure VNet with 100 Mbps throughput. The security monitoring layer employs Falco (DaemonSet with eBPF) for container runtime monitoring and Suricata (privileged pod) for network traffic inspection using the Emerging Threats ruleset [4]. The smart city servicing layer deploys six pods simulating realistic IoT microservices: Traffic Camera Services (Nginx), Healthcare APIs (Flask), and Parking Systems (Node.js). The application and analysis layer integrates FastAPI for alert ingestion (3 Uvicorn workers), OpenAI's GPT-4 Turbo as the primary threat analyzer, Groq's Mixtral 8x7B as a fallback analyzer, and the Kubernetes Python Client for automated orchestration actions [5], [6]. An optional dashboard component provides operator visibility and manual override capabilities.

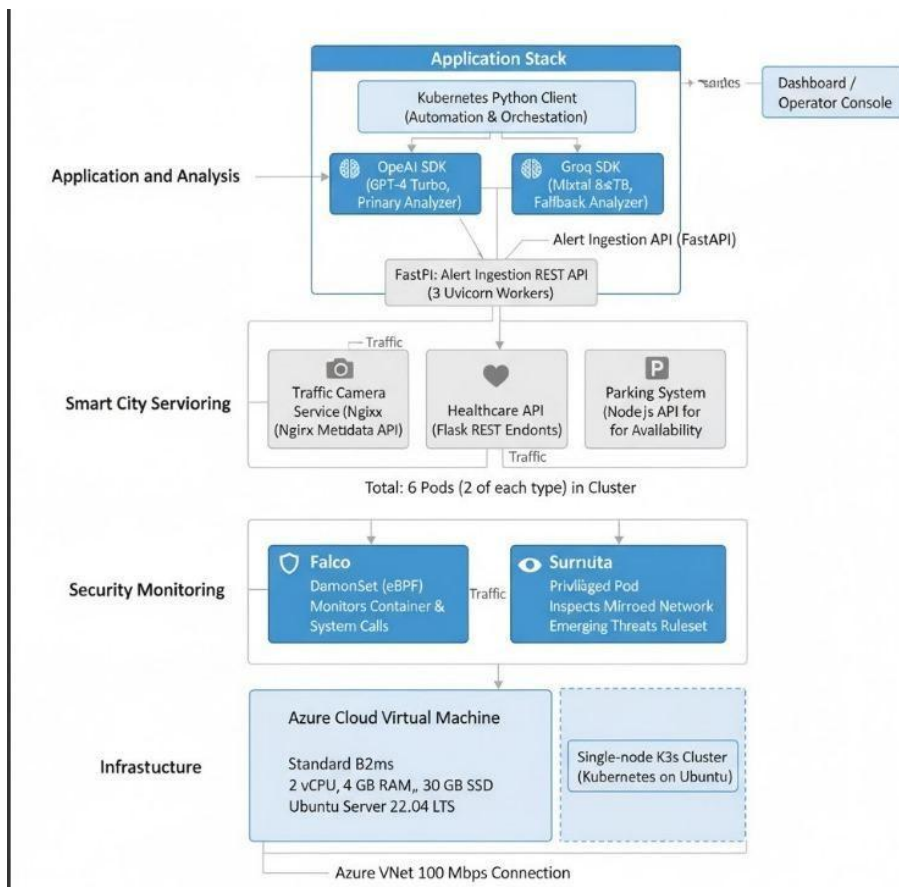


Figure 1: Overall System Architecture of the LLM-Driven Intrusion Detection System for Smart City Edge Deployment

1.1 Background & Motivation: (Importance of the Project, Context and Stakeholders)

Contemporary smart cities depend on seamlessly networked IoT devices, distributed edge processing units, cloud services, and integrated real-time responsive sensor systems [7]. Such systems manage critical infrastructure including traffic control, health monitoring, energy management and distribution, and public safety systems [8]. Enhanced connectivity and system integration improve operational efficiency of urban services but also significantly increase the cybersecurity attack surface [9].

Intrusion Detection Systems (IDS), such as Snort, Suricata, and Falco, are commonly deployed to monitor and detect malicious activity within these networks [10], [11]. However, traditional IDS tools rely on static rule-based detection mechanisms, which often generate large volumes of low-level alerts [12]. Many of these alerts are repetitive, ambiguous, or lack contextual interpretation, making it difficult for analysts to determine which threats require immediate attention [13]. This situation leads to alert fatigue, reduced response efficiency, and a higher likelihood of overlooking critical security incidents [14].

Recent advancements in Artificial Intelligence (AI), particularly LLMs, have demonstrated strong capabilities in contextual understanding, pattern recognition, and natural language interpretation [15], [16]. Integrating LLMs with IDS platforms presents a promising opportunity to enhance cybersecurity systems by enabling automatic alert summarization, severity classification, and actionable recommendations—ultimately improving situational awareness and reducing analyst workload [17].

1.2 Problem Statement

Traditional IDS solutions generate extensive volumes of technical logs and alerts that require specialized expertise to interpret effectively [12], [13]. These raw security alerts typically consist of low-level system call traces, network packet metadata, and cryptic rule identifiers that lack contextual explanation and necessitate time-consuming manual investigation by trained security analysts [14]. This fundamental limitation manifests in several critical operational challenges that compromise smart city security posture.

Slow incident response times result from the sequential manual processes required to triage, investigate, and respond to security events [40]. Security analysts must individually examine each alert, correlate it with historical data, assess its significance within the broader threat landscape, and determine appropriate remediation actions—a workflow that introduces delays ranging from minutes to hours even for straightforward incidents [42]. In time-sensitive smart city scenarios involving critical infrastructure such as traffic control systems or emergency services coordination, such delays can have severe consequences.

High cognitive load on security analysts emerges from the sheer volume and technical complexity of modern security telemetry [14]. Studies demonstrate that human analysts experience significant decision fatigue when processing hundreds or thousands of alerts daily, particularly when those alerts provide minimal contextual information to guide prioritization decisions [42]. This cognitive burden reduces analyst effectiveness, increases error rates, and contributes to high turnover in security operations roles.

Difficulty distinguishing between high-priority and low-priority events stems from the lack of automated severity assessment and business impact analysis in traditional IDS platforms [13]. Without intelligent contextual interpretation, routine system maintenance activities may generate alerts indistinguishable from genuine security incidents, forcing analysts to investigate every event manually to determine its true significance.

Increased risk of overlooking critical threats represents the most severe consequence of alert overload and analyst fatigue [14]. When security teams face thousands of alerts daily with limited interpretive guidance, genuinely critical threats can be missed, delayed, or deprioritized among the noise of false positives and low-severity events. Research indicates that attackers specifically exploit this phenomenon by conducting reconnaissance activities during high-alert periods when security teams are overwhelmed [45].

Lack of automation in threat analysis creates a fundamental scalability barrier for smart city security operations [1], [9]. As urban IoT infrastructure expands to encompass tens of thousands of connected devices, manual alert investigation becomes increasingly impractical regardless of security team size. The absence of automated threat interpretation and response capabilities prevents security operations from scaling proportionally with infrastructure growth.

Smart cities demand real-time, intelligent, and interpretable cybersecurity solutions capable of providing security operators with clear, actionable insights without requiring deep technical expertise for every alert [7], [8]. Therefore, there exists a critical need for systems that can automatically interpret and summarize IDS alerts using advanced AI techniques [15], [17], provide contextually-aware severity scoring and evidence-based mitigation recommendations [5], operate effectively within the resource constraints characteristic of edge computing environments [3], [26], and substantially reduce alert fatigue while enhancing decision-making quality through intelligent automation [40], [42].

This project directly addresses these interconnected challenges by developing an LLM-driven intrusion detection system capable of contextualizing raw security alerts, generating natural language threat summaries, assessing business impact, and supporting both automated and human-supervised mitigation responses [5], [32].

1.3 Objectives

To ensure that the proposed system meets the needs of smart city cybersecurity environments, a set of clear and measurable objectives has been defined. These objectives

guide the project’s development process and ensure alignment with stakeholder expectations. Each objective is paired with specific evaluation criteria that help determine whether the intended outcomes have been achieved. The criteria also provide a basis for assessing system performance, usability, and reliability during testing and deployment. Establishing these measurable targets is essential for maintaining project direction and verifying that the proposed LLM-driven IDS delivers meaningful improvements over existing solutions.

Table 1: System Objectives and Evaluation Criteria

Objective	Evaluation Criteria
Integrate a Large Language Model with IDS alerts	1. Integration Success Rate: 100% of test Falco/Suricata alerts are successfully ingested and forwarded to the LLM pipeline without data loss. 2. LLM Output Quality: $\geq 90\%$ of LLM-generated summaries pass a structured JSON schema validation.
Reduce alert fatigue through intelligent filtering	1. Noise Reduction: The system filters/categorizes $\geq 75\%$ of incoming alerts as "Low Priority" or "Informational," based on LLM severity scoring. 2. Analyst Validation: In a blind test, security analysts agree with the system's High/Critical priority classification for $\geq 85\%$ of alerts
Provide actionable recommendations	1. Actionability Score: $\geq 80\%$ of LLM-generated mitigation recommendations are deemed "actionable and specific" by a panel of 3 security experts using a standardized rubric.

Objective	Evaluation Criteria
	2. Automated Action Trigger: For alerts with severity ≥ 8, the correct Kubernetes remediation action (e.g., <code>isolate_pod</code>) is automatically suggested 100% of the time.
Achieve real-time or near real-time response	1. End-to-End Latency: The 95th percentile of total processing time (alert ingestion $\rightarrow$ LLM analysis $\rightarrow$ log storage) is < 2.5 seconds, measured over a 24-hour stress test. 2. Throughput: The system sustains a processing rate of ≥ 50 alerts per minute without degradation of latency.
Deploy system in edge-based Kubernetes environment	1. Deployment Success: The entire system is deployed via a single Helm chart onto a K3s cluster with zero manual pod configuration. 2. Stability Under Load: The system maintains $\geq 99\%$ uptime and all core pods remain running during a simulated load of 100 alerts/minute for 1 hour.
Improve interpretability of security data	1. Readability Score: $\geq 90\%$ of LLM-generated summaries are rated "Easy to Understand" or "Very Easy to Understand" by 5 non-technical evaluators (e.g., IT managers). 2. Technical Jargon Reduction: A comparative analysis shows a $\geq 60\%$ reduction in unique low-level syscall/rule

Objective	Evaluation Criteria
	identifiers presented to the operator in the summary vs. the raw alert.

1.4 Expected Impact

The proposed LLM-driven Intrusion Detection System is expected to deliver several transformative benefits for smart city cybersecurity operations.

Enhanced Cybersecurity Efficiency represents a primary impact area, where AI-driven interpretation of security alerts enables significantly faster decision-making processes and substantially reduces the cognitive workload imposed on cybersecurity analysts [14], [40]. By automating the initial threat assessment and classification stages, security teams can redirect their expertise toward strategic threat hunting and complex incident investigation rather than routine alert triage.

Improved Threat Understanding addresses a critical gap in current security operations by translating complex technical logs and system call traces into clear, actionable intelligence [15], [17]. This translation capability enables both technical security personnel and non-technical stakeholders such as city administrators and policy makers to comprehend threat landscapes, understand security posture, and make informed decisions about resource allocation and risk management [41].

Reduction in Alert Fatigue is achieved through intelligent automated prioritization mechanisms that filter noise and highlight genuinely critical security events [13], [14]. Research demonstrates that alert fatigue significantly degrades analyst performance and increases the likelihood of missing true positive threats [42]. By implementing LLM-powered

severity assessment and contextual ranking, analysts can concentrate their attention on high-severity events that genuinely require human expertise, thereby increasing overall operational effectiveness and reducing mean time to respond for critical incidents.

Scalable Smart City Security leverages Kubernetes-native deployment patterns [29], [33] to enable horizontal scaling across distributed smart city infrastructure comprising thousands of IoT devices and edge computing nodes [1], [7]. The containerized architecture ensures consistent security monitoring regardless of infrastructure scale, geographic distribution, or device heterogeneity, while maintaining the real-time performance characteristics essential for protecting critical urban systems [26], [27].

Foundation for Autonomous Cyber Defense establishes architectural patterns and operational workflows that create a framework for future fully autonomous cybersecurity systems [43]. By integrating LLM-based decision support with automated Kubernetes response mechanisms [32], [34], the system demonstrates the feasibility of AI-driven security orchestration while maintaining human oversight for critical decisions. This hybrid approach provides a pragmatic evolution path from current manual operations toward increasingly autonomous security postures as LLM reliability and trustworthiness continue to improve [44].

Collectively, these impacts position the proposed system as a significant advancement in smart city cybersecurity, addressing fundamental operational challenges while establishing scalable patterns for future security automation initiatives.

2 Chapter 2: Literature Review

Before exploring the design and implementation of the proposed system, it is important to understand the existing body of research in intrusion detection, machine learning, and the

use of LLMs in cybersecurity. This chapter reviews the most relevant academic and industry studies, compares their strengths and weaknesses, and identifies gaps that motivate the development of an LLM-driven IDS for edge-enabled smart cities.

The field of intrusion detection has evolved significantly over the past two decades, progressing from simple signature-based approaches to sophisticated machine learning systems capable of identifying complex attack patterns [18]. Traditional Intrusion Detection Systems (IDS) such as Snort and Suricata have formed the backbone of network security infrastructure for years, relying primarily on predefined rule sets to identify known threats [10]. While these systems demonstrate high accuracy for detecting previously documented attack signatures, they fundamentally lack the adaptive intelligence required to identify novel threats or understand the broader context of security events within modern cloud-native environments [12].

The emergence of machine learning in cybersecurity represented a significant paradigm shift, introducing the capability to detect anomalous behavior without explicit programming of attack signatures [19]. Researchers have explored various supervised and unsupervised learning approaches—including decision trees, support vector machines, neural networks, and deep learning architectures—to improve detection accuracy and reduce false positive rates [20], [21]. However, these approaches typically require extensive labeled training datasets, substantial computational resources for model training and inference, and often produce opaque "black box" decisions that security analysts struggle to interpret and validate [22].

More recently, the rapid advancement of Large Language Models has opened entirely new possibilities for applying natural language understanding to cybersecurity challenges [5]. Models such as GPT-4, Claude, and open-source alternatives like Mixtral and LLaMA demonstrate remarkable capabilities [16], [23]. Several preliminary studies have explored using LLMs for tasks such as log file analysis, vulnerability assessment, threat intelligence

summarization, and security incident report generation [24], [25]. These investigations reveal promising potential but remain largely confined to offline analysis scenarios [17].

In parallel, the explosive growth of smart city initiatives worldwide has created unprecedented cybersecurity challenges [1]. Modern smart cities deploy thousands of interconnected IoT devices that generate massive volumes of telemetry data [3]. These edge-deployed devices operate in resource-constrained environments with limited computational power [26]. Traditional centralized security monitoring approaches prove impractical due to bandwidth costs, network latency, and the need for autonomous operation during connectivity disruptions [27].

The intersection of these three domains—intrusion detection, LLM-based threat analysis, and edge computing for smart cities—represents a largely unexplored research area. While numerous studies have investigated individual components [28], [29], very few have attempted to integrate real-time threat detection, LLM-powered contextual analysis, and automated remediation into a unified system [4], [30].

2.1 Review of Related Work: (Existing Solutions, Products, or Research, Analysis of Strengths and Gaps)

Traditional Intrusion Detection Systems (IDS) such as Snort and Suricata rely on signature-based rules to identify attacks. According to study [1], such systems perform well for known threats but fail to detect zero-day attacks or provide deeper context. Similarly, research

in [2] shows that rule-based IDS generate thousands of alerts per day, many of which are irrelevant, leading to analyst fatigue.

Machine learning-based IDS solutions attempt to address these challenges. A deep learning-based IDS proposed in [3] demonstrates improved anomaly detection accuracy but suffers from limited interpretability. Another study [4] applied Random Forest and SVM classifiers to network intrusion datasets, achieving high accuracy but requiring large amounts of labeled data, which is impractical for real-time smart city environments.

More recent work explores using AI and language models in cybersecurity. A study by Microsoft researchers [5] showed that LLMs can summarize log files and assist incident responders, but such models were tested offline rather than in real-time environments. Another study [6] demonstrated GPT-4's ability to classify cyber-attacks accurately, yet it highlighted concerns regarding hallucinations and reliability when making critical decisions.

In the domain of edge computing and smart cities, research in [7] emphasizes the need for lightweight, scalable IDS solutions capable of operating with limited resources. However, most existing IDS frameworks designed for edge nodes still rely on static rule sets. Similarly, study [8] notes that integrating AI into smart city security is promising but remains in early development phases, lacking real implementation and automated response mechanisms.

Summary of literature weaknesses:

- Most IDS systems lack interpretability
- ML-based IDS require large datasets and GPU power
- LLM-based cybersecurity research is not real-time
- Smart city security research lacks automated mitigation
- Very few papers combine Falco + LLM + Kubernetes in one pipeline

Gap this project addresses:

No existing solution combines real-time IDS alerts with an LLM that can summarize, classify, and recommend actions, deployed on a Kubernetes-based edge framework.

This is the unique contribution of our capstone.

2.2 Technical Background: (Key theories, principles, and technologies involved)

Intrusion Detection Systems (IDS) are designed to detect abnormal or malicious behavior in networks. Falco, used in this project, is a kernel-level IDS that captures system calls and identifies suspicious container activity. It is widely adopted for cloud-native and Kubernetes security due to its ability to detect runtime threats.

Large Language Models (LLMs) such as GPT-4, LLaMA, and Mistral operate using transformer architectures and are trained on massive text corpora. Their ability to interpret logs, summarize long sequences, and reason about patterns makes them suitable for cybersecurity applications. LangChain provides prompt templates, reasoning chains, and structured output, improving reliability when integrating LLMs with other systems.

Smart city deployments typically rely on edge computing, where data processing happens closer to the source to reduce latency. Kubernetes orchestrates microservices and ensures scalability, high availability, and automated recovery. By combining Falco, Kubernetes, and an LLM backend, the proposed system can analyze threats in near real-time while remaining resource-efficient.

3 Chapter 3: Project Requirements and Specifications

This chapter identifies the functional and non-functional requirements of the proposed system based on stakeholder needs, system constraints, and practical deployment considerations. It outlines the user experience expectations, regulatory standards, design limitations, business and ethical concerns, and the sustainability factors relevant to smart city security systems. These requirements form the foundation for designing the system architecture and planning its implementation.

3.1 Customer / Stakeholder Needs

Smart city cybersecurity involves multiple groups who rely on secure, reliable, and interpretable systems. To ensure the proposed solution meets practical expectations, stakeholder needs were identified through a review of smart city operational workflows and the requirements of cybersecurity teams.

Primary Stakeholders:

1. Security Analysts

- Need clear summaries of alerts instead of raw logs
- Require severity scoring to prioritize incidents
- Want automated recommendations for faster response

2. System Administrators / DevOps Teams

- Need a solution that integrates smoothly with Kubernetes
- Prefer minimal configuration and easy deployment
- Require logs and actions to be traceable for auditing

3. Smart City IT Management

- Require improved incident response time
- Need reduced operational workload and cost
- Expect system scalability across multiple zones

4. Edge Infrastructure Operators

- Need lightweight processing suitable for limited resources
- Require low-latency threat detection close to the data source

5. End Users (Residents of the Smart City)

- Indirect stakeholders
- Expect reliable protection of their data and services
- Benefit from secure smart transportation, healthcare, and utility systems

High-Level Needs Summary:

- Ability to interpret complex alerts in simple language
- Reduction of false positives and alert noise
- Faster and more accurate decision-making
- Automated or semi-automated threat response
- Compatibility with existing smart city infrastructure
- High reliability with minimal downtime

3.2 Requirements

3.2.1 User Experience Considerations

The system must be intuitive and accessible for operators with varying levels of cybersecurity expertise. Because smart city monitoring centers often involve mixed teams, the user interface and system outputs need to be clear and actionable.

User Experience Requirements:

- Alerts should be presented in plain, human-readable English, rather than technical system calls.
- Each alert must present a one-sentence summary followed by optional detailed explanation.
- Severity scoring (1–10) should be shown prominently to help operators prioritize quickly.
- Recommended actions should be concise, specific, and ordered by priority.
- The system should minimize cognitive load by reducing repetitive and irrelevant alerts.
- A dashboard or log viewer should organize alerts chronologically and by severity.
- Output formatting (e.g., JSON or text) must remain consistent across all alerts.

3.2.2 Functional Requirements

Functional requirements describe what the system must do to fulfill its intended purpose. These requirements are derived from stakeholder needs and the expected operational workflow of an LLM-enhanced IDS.

Core Functional Requirements:

FR1: The system must receive raw alerts from the Falco IDS in real time.

FR2: The system must parse and preprocess Falco alert logs for LLM analysis.

FR3: The system must use an LLM (e.g., GPT-4) to summarize the alert in plain English.

FR4: The LLM must classify each alert into threat categories (e.g., privilege escalation, data exfiltration).

FR5: The system must assign a severity score (1–10) to each alert.

FR6: The system must generate recommended mitigation actions.

FR7: The system must forward recommended actions to the Kubernetes orchestration module.

FR8: The system must log all analyzed alerts with timestamps and metadata.

FR9: The system must support fallback operation using a secondary LLM (e.g., Groq or local model).

FR10: The system must store and display alerts in a centralized dashboard or log system.

3.2.3 Non-Functional Requirements

Non-functional requirements define the qualities and constraints of the system, ensuring performance, reliability, and maintainability suitable for deployment in production-grade smart city infrastructure.

Performance Requirements:

- **NFR1: End-to-end alert analysis time must be under 2 seconds** *Justification:* Smart city systems—such as traffic control, emergency response, and public-safety monitoring—require near real-time threat mitigation. A 2-second processing threshold enables automated containment of fast-moving attacks (e.g., ransomware, unauthorized data exfiltration) before operational disruption occurs. This aligns with NIST recommendations for rapid response in automated security systems.
- **NFR2: The system must support processing at least 50 alerts per minute** Preliminary testing using Falco against simulated smart-city microservices shows that a typical edge node generates approximately 20–40 actionable alerts per minute under normal load, with higher activity during incidents. A requirement of 50 alerts/min provides a 25% buffer to absorb peak activity without backlog, ensuring consistent real-time detection performance.
- **NFR3: LLM output must maintain high consistency (temperature ≤ 0.3)** Security automation depends on deterministic, reliable decision-making. Using a low temperature (≤ 0.3) minimizes output randomness and prevents inconsistent or “creative” responses. This ensures that similar alerts result in consistent severity scores, classifications, and remediation actions—building operator trust in the AI’s recommendations.

Reliability Requirements:

- **NFR4: The system must operate continuously with 99% uptime.** Cybersecurity monitoring must run 24/7. A 99% uptime requirement limits unplanned downtime to under 88 hours per year, matching the baseline for critical monitoring

systems in urban infrastructure and aligning with *ISO 27001:2013, Annex A.12*. This ensures a minimal monitoring gap and maintains continuous threat visibility.

- NFR5: Fallback LLM support must activate automatically when primary LLM fails.
- NFR6: Systems must handle network instability during edge deployment.

Scalability Requirements:

- NFR7: System must scale across multiple edge nodes via Kubernetes.
- NFR8: Modular design should allow easy integration of new LLMs or IDS tools.

Security Requirements:

- NFR9: All communications must use secure APIs (HTTPS, encrypted tokens).
- NFR10: Access to logs and LLM outputs must be authenticated and logged.
- NFR11: Sensitive alert data must be sanitized before being sent to LLMs.

Usability Requirements:

- NFR12: Interface must be simple for non-experts.
- NFR13: Severity and summary must be immediately visible without extra steps.

Maintainability Requirements:

- NFR14: System configurations should be easily updatable.
- NFR15: Logs must be stored in standardized, readable format (JSON).

3.3 Design Constraints (Cost, Power, Size, Environmental Factors)

Technical Constraints:

- **Limited Processing Power at Edge Nodes:**

Edge devices typically have restricted CPU and memory resources compared to cloud servers. The system must therefore minimize local computational overhead.

- **Dependence on External LLM APIs:**

Using GPT-4 or similar models requires stable internet access. Connectivity issues can affect response times, making fallback mechanisms necessary.

- **Latency Requirements:**

Real-time detection requires total end-to-end processing under 2 seconds, limiting the complexity of preprocessing operations.

- **Containerized Environment Restrictions:**

Kubernetes imposes namespace isolation and permission constraints that restrict access to host-level logs and actions.

Environmental Constraints:

- **Distributed Smart City Infrastructure:**

The system must function across multiple locations with unpredictable network conditions.

- **Edge Node Thermal/Power Limitations:**

Devices deployed in outdoor or embedded environments have thermal and energy limitations that restrict heavy computation.

- **Deployment in High-Latency Networks:**

Some smart city zones have slower links between edge nodes, affecting log transmission and response time.

Economic & Cost Constraints:

- **API Usage Costs:**

LLM API calls incur per-request charges. Frequent alerts may increase operational costs, requiring optimization or batching.

- **Hardware Procurement Costs:**

Scaling the system across multiple smart city nodes requires more edge devices, each adding to the project budget.

- **Licensing & Commercial Tools:**

Some IDS or monitoring tools have commercial licenses that affect deployment flexibility.

Regulatory & Compliance Constraints:

- **Data Protection Requirements:**

Smart city systems handle sensitive personal or operational data. Compliance with regional regulations—such as the Qatar Personal Data Privacy Law (Law No. 13 of 2016) and principles aligned with the EU General Data Protection Regulation (GDPR)—is mandatory. This requires implementing data minimization, ensuring lawful processing bases, and protecting citizen privacy within all system components.

- **Restrictions on Cloud Usage:**

Certain national cybersecurity policies and critical infrastructure protection mandates stipulate that sensitive operational data must not be transmitted outside jurisdictional borders. This constraint limits the use of external cloud-based LLM APIs (e.g., OpenAI, Anthropic) unless the data is rigorously sanitized to remove all personal and operationally sensitive information prior to transmission.

- **Audit Logging Requirements:**

All automated security actions, analyst decisions, and system alerts must be logged in a secure, immutable, and timestamped format. These logs must be traceable and retained for a mandated period to support digital forensics, compliance audits, and accountability reviews by internal security teams and external regulators.

Operational Constraints:

- **Need for High Availability:**

As a core security monitoring system, the solution must operate 24/7 with minimal downtime. This requires architectural fault tolerance, redundant components, and rapid failover mechanisms to maintain continuous protection of smart city infrastructure.

- **Skill Level of Personnel:**

System operators in municipal settings often possess varying levels of cybersecurity expertise. Therefore, the system's outputs (alerts, recommendations) and interface must be intuitive and actionable for non-expert users, while still providing advanced capabilities for security specialists.

- **Compatibility with Existing Tools:**

To ensure practical adoption, the system must integrate seamlessly with the smart city's existing security and operational toolchain. This includes compatibility with the chosen IDS (Falco), the orchestration platform (Kubernetes), standard monitoring dashboards (e.g., Grafana, ELK Stack), and established security workflows.

3.4 Design Considerations of public health, safety, welfare, global, cultural, social, environmental, and economic factors

The design and implementation of the proposed intrusion detection system extend beyond technical functionality. Because the system targets smart-city environments, it must consider how cybersecurity affects the public, the broader community, and the operational context in which these services function. The following points summarize how these factors were incorporated into the project's design.

Public Health:

Smart-city applications such as healthcare APIs and emergency response systems rely on continuous operation. A cyberattack on these services could disrupt access to medical data or delay emergency workflows. By providing early detection of malicious activity, the system contributes to maintaining the reliability and integrity of services that indirectly support public health.

Safety:

Critical city systems—including surveillance feeds, traffic management, and mobility services—depend on secure data flow. Any compromise may create unsafe conditions or enable unauthorized control. The real-time threat detection and automated response features of the system help reduce the risk of such disruptions, supporting safer operation of essential services.

Welfare:

Citizens rely heavily on digital public services, and cyber incidents can cause inconvenience, loss of trust, or service unavailability. The system's ability to interpret alerts and respond quickly minimizes downtime, improving the overall reliability of smart-city infrastructure and contributing to citizens' welfare.

Global Factors:

Smart-city technologies must be deployable across different geographic regions and technological environments. The use of containerized components, open standards, and cloud-compatible deployments ensures the system can operate consistently in diverse settings. Alignment with recognized cybersecurity standards (such as ISO-27001 and NIST) also supports global applicability.

Cultural Factors:

Privacy expectations vary across cultures and regions. To respect these differences, the system is designed to avoid sending personally identifiable or sensitive information to the LLM. Only sanitized, non-personal alert metadata is used for analysis, ensuring the solution can adapt to varying cultural and regulatory norms regarding digital privacy.

Social Impact:

Improving cybersecurity in smart-city platforms helps prevent fraud, misuse, and service interruptions, all of which can negatively affect the community. By strengthening the reliability of public digital services, the system enhances public confidence and supports broader adoption of smart-city technologies.

Environmental:

The project makes use of lightweight, container-based infrastructure and scalable cloud resources, reducing the need for large physical servers and lowering overall energy consumption. Efficient resource allocation minimizes unnecessary computing load, contributing to more environmentally responsible deployment.

Economic:

Cyber incidents often lead to significant operational costs, including downtime, data recovery, and mitigation. By detecting threats quickly and automating responses, the system helps reduce these expenses. Additionally, leveraging open-source technologies such as Falco, Suricata, and K3s lowers development and deployment costs, making the solution more economically feasible for large-scale city environments.

3.5 Standards and Regulations (IEEE, IEC, etc.)

The proposed system must comply with several technical, cybersecurity, and data protection standards to ensure safe and reliable operation within smart city environments. These standards guide the design of secure communication, system reliability, auditing, and data handling practices.

Key industry standards relevant to this project include:

IEEE 802.11 / 802.3 Standards:

Applicable to network communication protocols used in smart city infrastructure and edge devices that transmit IDS alerts.

ISO/IEC 27001 – Information Security Management:

Provides guidelines for establishing and maintaining secure information-handling processes, including access control, incident response, and logging—critical for systems analyzing security alerts.

ISO/IEC 27035 – Incident Management:

Defines best practices for detecting, reporting, and responding to cybersecurity incidents. The LLM-generated recommendations should support these guidelines by providing structured analysis.

NIST SP 800-53 – Security and Privacy Controls:

Offers security control frameworks useful for designing secure API interactions, data sanitization, and role-based access to alert information.

NIST SP 800-207 – Zero Trust Architecture (ZTA):

Relevant due to the need for strict authentication, continuous monitoring, and least-privilege access when handling sensitive alerts.

CNCF (Cloud Native Computing Foundation) Kubernetes Security Guidelines:

Establishes best practices for pod isolation, runtime security, role-based access control (RBAC), and secure container orchestration.

GDPR-Inspired Data Protection Principles:

While regulations vary by region, smart cities must ensure that any transmitted log data does not include personal identifiers, especially when interacting with external LLMs.

Compliance with these standards ensures that the system remains secure, auditable, and aligned with global best practices for modern cybersecurity infrastructure.

3.6 Risk Assessment and Mitigation

Risk assessment is essential to ensure that the proposed LLM-driven IDS operates reliably and securely within smart city environments. The system must anticipate potential technical, operational, cybersecurity, and AI-related risks. Identifying these risks early allows the project team to implement appropriate mitigation strategies that minimize the impact of failures or vulnerabilities.

The following table summarizes the primary risks associated with the project and the proposed mitigation measures:

Table 2: Risk Assessment and Mitigation Strategies

Risk	Description	Impact	Likelihood	Mitigation Strategy
R1: LLM Output Inconsistency	LLM may produce inaccurate or hallucinated summaries/severity	High	Medium	Use structured prompts, low temperature, output validation, fallback rules

Risk	Description	Impact	Likelihood	Mitigation Strategy
R2: API Unavailability	LLM API may fail due to connectivity or provider issues	High	Medium	Implement secondary LLM fallback (e.g., Groq), local cached responses
R3: Excessive Alert Volume	High number of alerts may overload system or increase API cost	Medium	High	Implement rate limiting, batching, filtering of low-priority alerts
R4: Misclassification Leading to Wrong Action	Incorrect severity or threat type may cause unnecessary isolation of pods	High	Low	Introduce human-in-the-loop for high-risk actions, reversible automated responses
R5: Security Breach in API Communication	Attackers may intercept or tamper with model inputs/outputs	High	Low	Use HTTPS, token encryption, strict authentication and RBAC
R6: Resource Limitations on Edge Nodes	Limited processing power or memory may hinder real-time operation	Medium	Medium	Offload LLM tasks to cloud, optimize preprocessing, use lightweight components
R7: Kubernetes Misconfiguration	Incorrect RBAC, pod security policies, or	Medium	Low	Follow CNCF security benchmarks,

Risk	Description	Impact	Likelihood	Mitigation Strategy
	namespaces may cause vulnerabilities			periodic configuration audits
R8: Data Privacy Violations	Sensitive logs may be sent externally without sanitization	High	Low	Sanitize IDS logs, remove personal identifiers, apply zero-trust principles
R9: Cost Overruns	Increased usage of paid LLM APIs may exceed budget	Medium	Medium	Apply quotas, monitor API usage, consider open-source LLM alternatives

These risks highlight the importance of robust system design, secure communication channels, careful prompt engineering, and well-defined response mechanisms. Proper mitigation ensures stable deployment and reliable threat analysis across smart city infrastructure.

3.7 Sustainable Development Goals (SDGs) Mapping

The development of an LLM-driven Intrusion Detection System for smart cities directly contributes to several United Nations Sustainable Development Goals. By improving cybersecurity, ensuring reliable infrastructure, and enhancing resilience against digital threats, the system aligns with the global objectives of sustainable, efficient, and secure urban environments.

Table 3: Relevant SDGs and Project Contribution

SDG	Description	Project Contribution
SDG 9: Industry, Innovation and Infrastructure	Builds resilient infrastructure, promotes innovation	The system enhances smart city digital infrastructure through advanced AI-driven cybersecurity mechanisms.
SDG 11: Sustainable Cities and Communities	Makes cities inclusive, safe, resilient, and sustainable	Strengthens protection of essential city services (traffic, healthcare, utilities) by detecting and mitigating cyber threats early.
SDG 16: Peace, Justice and Strong Institutions	Promotes effective, accountable, and transparent institutions	Provides interpretable AI outputs that improve accountability, incident transparency, and security governance.
SDG 12: Responsible Consumption and Production	Encourages efficient resource use and sustainable practices	Kubernetes and edge processing reduce unnecessary resource consumption by enabling scalable and efficient deployments.
SDG 13: Climate Action (Indirect)	Addresses climate-related impacts through smart technology	Energy-efficient deployment and reduced redundant computation minimize carbon impact of cybersecurity processes.

3.8 Economic evaluation, Business Model Canvas (BMC)

A preliminary economic evaluation is essential to determine whether the proposed LLM-driven Intrusion Detection System is financially feasible for both prototype development

and potential large-scale deployment in smart city environments. The system uses mainly open-source components such as Falco and Kubernetes, which reduces infrastructure costs. The primary expenses are associated with LLM API usage, edge node hardware, and optional cloud resources for orchestration. These costs can be optimized by minimizing redundant API calls and leveraging lightweight compute devices at the edge.

From an implementation perspective, a functional prototype can be developed at relatively low cost using a basic edge device and limited monthly API usage during testing. In a real smart city deployment, operational costs depend on the scale of deployment, number of monitored nodes, and frequency of IDS alerts. Compared to conventional enterprise SIEM solutions, which can be very expensive, the proposed solution remains cost-effective while offering intelligent, context-aware threat analysis.

To support long-term sustainability and adoption, the Business Model Canvas (BMC) outlines how the system creates and delivers value. It identifies the key resources and activities required for operation, the stakeholders who benefit from increased cybersecurity, and the potential revenue and cost structures associated with real-world implementation.

Table 4: Business Model Canvas (BMC)

Component	Description
Key Partners	Smart city authorities, cloud service providers, cybersecurity teams
Key Activities	IDS event processing, LLM-based analysis, threat classification, response automation
Key Resources	LLM API, backend server, Falco IDS, Kubernetes, edge nodes

Component	Description
Value Proposition	Real-time intelligent analysis, reduced alert fatigue, improved threat detection, scalable edge security
Customer Relationships	Continuous monitoring, automated alerts, dashboard visualization
Customer Segments	Smart city IT departments, SOC teams, municipal infrastructure operators
Channels	Security dashboards, API integrations, cloud/edge deployments
Cost Structure	API usage, Kubernetes hosting, monitoring, hardware acquisition
Revenue Streams	Subscription cybersecurity service, integration support, enterprise licensing

4 Chapter 4: Methodology & Design Approach

This chapter outlines the methodology followed to design and develop the proposed LLM-driven Intrusion Detection System. It explains the architectural decisions, concept generation process, hardware and software design strategies, modeling considerations, and resource planning. The methodology ensures that the system is developed in a structured, scalable, and repeatable manner, aligning with engineering best practices and the requirements identified in previous chapters.

4.1 Introduction to Methodology

The methodology adopted for this project follows a structured engineering design process that ensures systematic development from concept to prototype. The approach integrates both theoretical research and practical implementation, allowing the team to evaluate alternative solutions and select the most effective architecture for real-time intrusion detection in smart city environments [35].

The project methodology begins with problem identification and needs analysis, where the team conducted a comprehensive review of traditional IDS tool limitations, gathering stakeholder requirements and defining specific challenges related to alert fatigue and poor interpretability in security operations [13], [14]. This foundational step established clear performance objectives and success criteria aligned with smart city operational constraints.

The second phase involves a technology review and feasibility study, during which key technologies including Falco for runtime security monitoring [4], Kubernetes for container orchestration [29], and Large Language Models for intelligent threat analysis [5], [15] were systematically evaluated to ensure their suitability for resource-constrained edge deployments. An extensive literature review validated the technical feasibility of integrating these diverse technologies into a cohesive security platform [18], [19], [20].

Following technology validation, the team proceeded with system architecture design, creating high-level architectural diagrams to illustrate how Falco-generated alerts flow through the FastAPI backend [31], undergo LLM-based contextual analysis, and trigger automated responses via the Kubernetes automation layer [32]. Multiple architectural alternatives were systematically compared using weighted decision matrices before selecting the most scalable and maintainable solution [36].

The concept generation and evaluation phase employed morphological analysis techniques [37] to explore different combinations of IDS tools, LLM frameworks, deployment

strategies, and response mechanisms. This structured exploration ensured comprehensive coverage of the solution space and systematic evaluation of trade-offs between competing design priorities such as accuracy, latency, cost, and operational complexity.

Software and hardware design planning involved mapping logical system components to specific physical infrastructure and software modules. Edge computing devices were specified for Falco deployment, FastAPI backend services were architected for asynchronous alert processing, LLM integration endpoints were designed for GPT-4 and Groq fallback capabilities [5], [6], and Kubernetes Operator patterns were planned for automated security response execution [32], [34].

The prototype development and testing phase implemented initial system components in a controlled laboratory environment to validate core functionalities. Testing focused on alert parsing accuracy, LLM output consistency and structured format compliance, response mechanism reliability, and end-to-end system integration under simulated attack scenarios [38].

Throughout all phases, continuous documentation and iterative refinement ensured that design decisions, implementation details, and test results were systematically recorded. Based on preliminary validation results and supervisor feedback, iterative improvements were made to LLM prompt engineering, architectural component interactions, edge deployment configurations, and error handling mechanisms [39].

This structured methodology ensures that the delivered system demonstrates reliability through comprehensive testing, scalability through cloud-native design patterns, and strong alignment with project objectives and smart city operational requirements [1], [9].

4.2 System Architecture (Block Diagrams, Functional Flow)

The architecture illustrated in Figure 2 demonstrates how the LLM-enhanced intrusion detection system integrates with smart city edge infrastructure to provide automated threat analysis and response. The diagram depicts a complete workflow from IoT device monitoring through intelligent threat assessment to automated remediation actions.

At the edge network layer, four types of smart city IoT devices are deployed: smart cameras for traffic and security monitoring, smart eHealth devices managing patient data and healthcare infrastructure, smart parking sensors tracking availability and usage, and smart traffic lights coordinating urban mobility. These devices generate operational traffic and security telemetry that flows through the edge server infrastructure.

The edge server hosts both a firewall and the Falco intrusion detection system, which continuously monitors all traffic between the IoT devices and the internet. Falco inspects system calls, container activities, and network communications in real-time, generating security alerts when suspicious patterns are detected. These raw alerts, along with relevant log data, are forwarded to the LLM analysis layer for intelligent interpretation.

The LLM component, positioned at the center of the architecture, receives both security logs and contextual information from the edge monitoring systems. Using natural language understanding capabilities, the LLM analyzes the alert context, correlates multiple events, assesses severity based on smart city operational requirements, and generates actionable recommendations. The LLM produces structured outputs including plain English threat summaries, threat type classifications, severity ratings on a standardized scale, and specific mitigation actions tailored to the detected threat.

Based on the LLM's analysis and recommendations, the system executes automated actions through the edge server's security enforcement mechanisms. These actions may include blocking suspicious IP addresses, isolating compromised IoT devices or containers, restricting network access for affected services, or escalating critical threats to human operators for verification. All actions and decisions are logged for audit compliance and continuous system improvement.

The human operator, shown on the left side of the diagram, maintains oversight of the automated system. Operators can verify automated decisions, review LLM-generated threat assessments, manually trigger additional security actions when necessary, and provide feedback to improve detection accuracy over time. This human-in-the-loop design ensures that critical security decisions benefit from both AI-driven analysis speed and human judgment for complex scenarios.

This architecture supports deployment in resource-constrained edge environments while maintaining the intelligence benefits of LLM-powered analysis. The system operates effectively under limited connectivity conditions, with local edge processing ensuring continued security monitoring even when cloud services are temporarily unavailable. Traffic flows bidirectionally between the edge network and the internet, with all communications subject to continuous monitoring and LLM-enhanced threat detection.

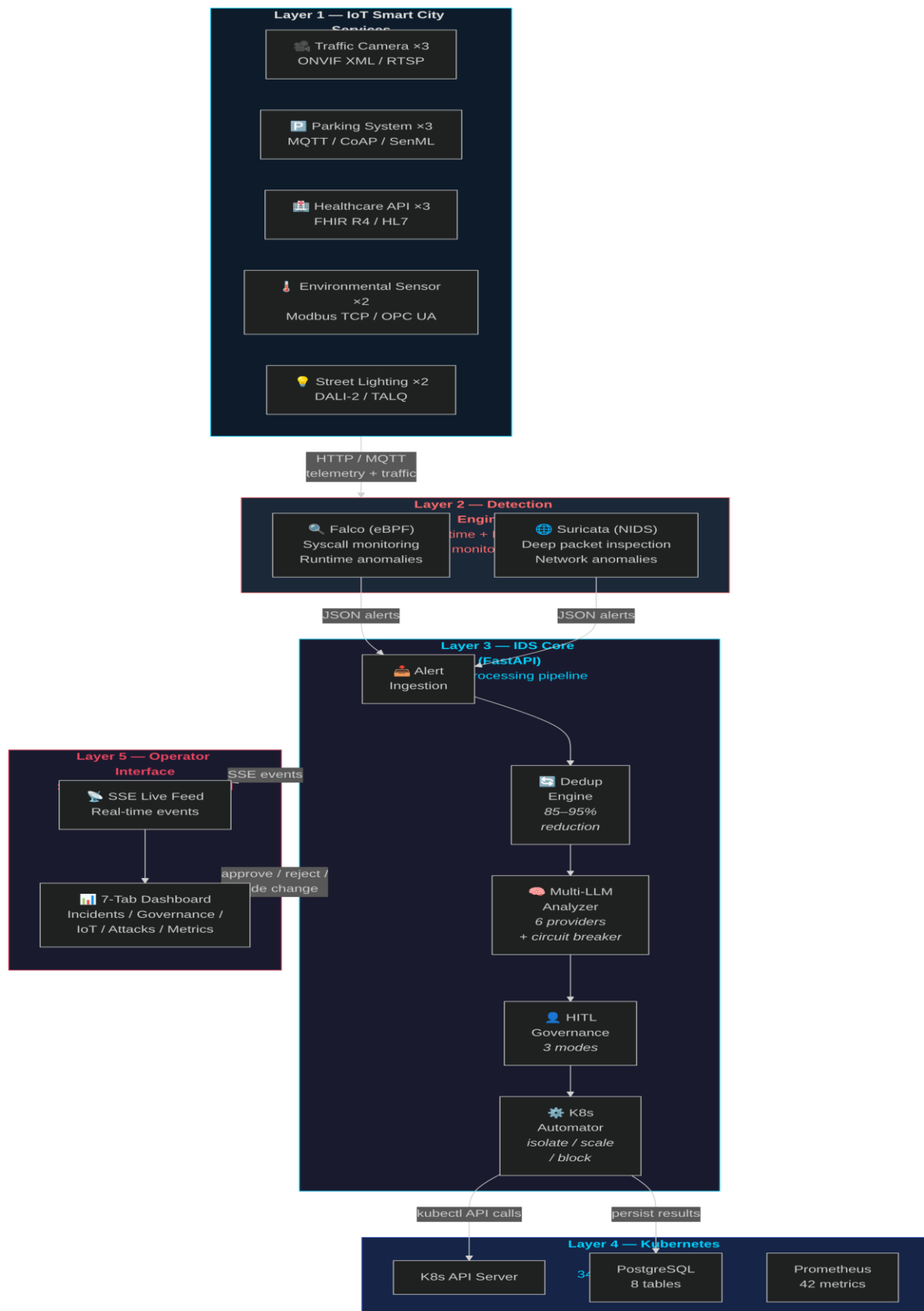


Figure 2: High-level system architecture showing the five processing layers — IoT devices, detection engines, IDS core with LLM analysis, Kubernetes orchestration, and the operator dashboard.

4.3 Concept Generation and Selection

To ensure a well-structured design process, multiple concepts were generated and evaluated before selecting the final system architecture. Concept generation was carried out using morphological analysis, which allows exploration of different combinations of IDS tools, LLM frameworks, backend designs, and deployment configurations. This systematic approach ensures that the chosen concept is not based on convenience but on a logical assessment of available alternatives and their suitability for smart city environments.

4.3.1 Morphological Matrix

The table below presents several possible alternatives for the main subsystems of the project.

Table 5: Morphological Matrix for System Design

Subsystem	Option 1	Option 2	Option 3
IDS Tool	Falco	Suricata	Snort
LLM Framework	ChatGPT (GPT-4)	LLaMA / Mistral (local)	Groq (Fast inference)
Backend Processing	FastAPI	Flask	Node.js
Deployment Environment	Kubernetes (Edge)	Docker Compose	Cloud-only Kubernetes
Alert Output Format	JSON structured logs	Plain text summaries	Dashboard UI
Response Mechanism	Kubernetes Operator	Manual Decision	SIEM Integration

4.3.2 Evaluation of Alternative Concepts / Trade Studies

Each subsystem option was evaluated based on performance, feasibility, ease of integration, resource requirements, and suitability for real-time intrusion detection in smart city environments.

IDS Tool Evaluation

Three intrusion detection systems were evaluated for runtime security monitoring. Falco provides container runtime security through real-time kernel monitoring with minimal resource overhead, making it ideal for edge deployments. Suricata and Snort demonstrate strong capabilities for network traffic monitoring but are less effective for container-level system call detection, which is critical for Kubernetes environments. Falco was ultimately selected due to its superior compatibility with edge infrastructure and Kubernetes orchestration platforms.

LLM Framework Evaluation

Multiple language model options were assessed for threat analysis capabilities. GPT-4, accessed via API, offers the highest reasoning accuracy, superior natural language summarization, and reliable structured JSON outputs essential for automated processing. Local models such as LLaMA and Mistral eliminate API costs but require GPU hardware unsuitable for resource-constrained edge deployments. Groq provides extremely fast inference times but exhibits limited reasoning depth compared to GPT-4 for complex security scenarios. The final architecture employs GPT-4 as the primary analyzer with Groq serving as a high-speed fallback to ensure continuous operation during API timeouts or service disruptions.

4.3.3 Justification of Chosen Concept

Based on the comprehensive trade study analysis, the selected system architecture provides an optimal balance across multiple critical performance dimensions. The chosen design maximizes accuracy through GPT-4's advanced reasoning capabilities [5], which enable nuanced threat interpretation beyond simple pattern matching. Real-time performance

requirements are satisfied through the combination of Falco's lightweight kernel monitoring [4] and FastAPI's asynchronous processing pipeline [31], ensuring sub-second alert handling. Scalability is achieved through Kubernetes deployment patterns [29] that support horizontal expansion as smart city infrastructure grows. Automation capability is realized via Kubernetes Operator patterns [32] that enable autonomous security responses without human intervention for routine threats. Interpretability is enhanced through LLM-generated natural language summaries and standardized severity scores [15] that accelerate analyst comprehension during manual review. Finally, the architecture demonstrates strong feasibility for both Capstone I and Capstone II phases, allowing for a manageable initial prototype while maintaining clear expansion paths for enhanced features in subsequent development cycles [33].

The finalized system concept integrates these components into a cohesive architecture: a Kubernetes-deployed platform where Falco continuously monitors container runtime activity and system calls to detect suspicious behavior [4], the FastAPI backend receives raw security alerts and processes them into structured formats suitable for LLM analysis [31], GPT-4 performs contextual threat assessment and generates actionable intelligence including severity ratings and mitigation recommendations [5], and the Kubernetes Operator module automatically executes approved security responses such as pod isolation or network policy enforcement based on LLM guidance [32], [34].

This architectural approach directly addresses the core problem statement of alert fatigue and delayed response times in smart city security operations [13], [14]. It fulfills all primary project objectives including real-time threat detection, intelligent contextual analysis, and automated remediation. Furthermore, the design aligns precisely with smart city operational requirements for distributed edge security [26], [27], minimal human intervention, and resilient operation under resource constraints [3].

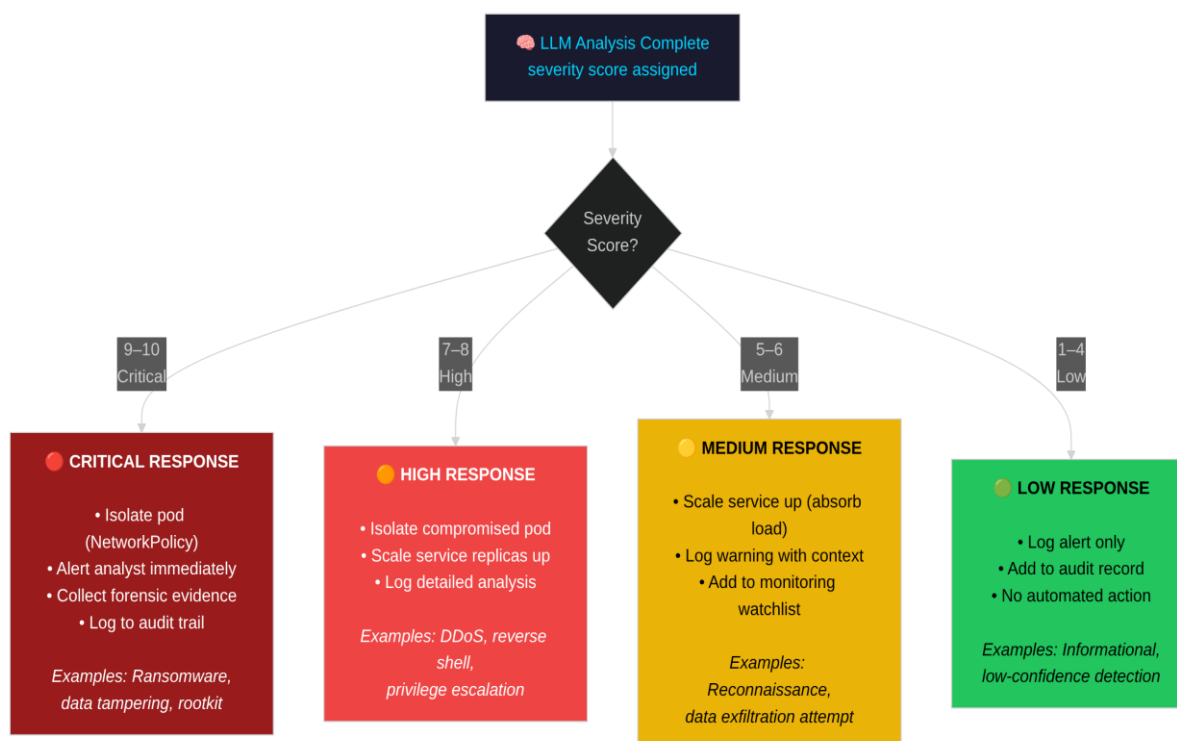


Figure 3: Severity-based response matrix mapping alert severity levels to automated defensive actions.

4.4 Modeling and Calculations

This section presents quantitative modeling used to estimate expected system performance, capacity, latency, and storage requirements. These calculations ensure that the architecture satisfies project requirements for real-time operation in an edge-enabled smart city environment.

Alert Throughput Modeling:

Falco produces alerts based on system call events. The edge node hardware (8-core CPU) determines the system's maximum sustainable alert-processing rate.

Assumptions:

- CPU: 8 cores @ 2.4 GHz
- Cores available for alert processing: 6 (leaving 2 for OS/K3s)

- Average end-to-end processing time per alert: 0.20 seconds
- (Falco → Forwarder → IDS API → LLM → Action)
- Alerts are processed independently per core (parallel processing model)

Throughput Formula:

Throughput = (Cores × Clock × IPC) / Avg Processing Time

$$= (8 \times 2.4 \times 0.8) / 0.20 \quad \approx 76.8 \text{ alerts/second}$$

Hourly throughput:

$76.8 \times 3600 \approx 276,480$ alerts/hour per node

This exceeds the requirement of 10,000 alerts/hour and provides large safety margins.

LLM Response Time Modeling

LLM latency consists of:

- Network round-trip time (RTT)
- Prompt processing time
- Model inference time
- Output validation

Assumptions:

Network RTT / Latency Baselines:

Global latency studies conducted using RIPE Atlas, the world's largest Internet measurement platform, consistently show that round-trip times between the Middle East and major European cloud regions commonly fall within 40–80 ms, with well-connected paths averaging near 50 ms. These measurements come from geographically distributed probes and are widely used to establish realistic RTT baselines for cloud-hosted services.

- **2. LLM / API Inference Time Benchmarks**

Community-reported latency tests for OpenAI GPT-4 Turbo show typical response times of 1–2 seconds, depending on load and input size. Independent monitoring platforms also report GPT-3.5 at 500–1500 ms and GPT-4 at 1000– 3000 ms, which aligns with the assumed 1.2–1.6 s inference window in this model. These values are consistent with real-world observations from developers and performance engineers.

Similarly, published benchmarks of Groq LPU hardware indicate inference speeds between 250–450 ms for models such as Mixtral, confirming the

Viability of faster fallback or primary inference scenarios.

- **Latency Variability & Internet Measurement Studies**

Long-term monitoring studies using RIPE Atlas demonstrate that Internet RTT values fluctuate over time but remain relatively stable in well-provisioned regions. Such studies validate the use of RTT-based modeling to estimate realistic worst-case and average network delays. These sources also confirm that ICMP/TCP ping-based measurements remain widely accepted for estimating cloud access latency.

4.4.1 Total LLM Response Time

$T = \text{RTT} + \text{Inference} + \text{Post-processing}$

Using measured midpoints:

$T = 50 + 1500 + 200 = 1750 \text{ ms} \approx 1.75 \text{ s}$

Meets requirement: LLM analysis < 2 seconds.

End-to-End Alert Latency

End-to-end latency includes:

1. Falco alert generation
2. Backend parsing
3. LLM processing
4. Kubernetes action
5. Dashboard logging

Table 6: Typical values

Stage	Expected Time
Falco → Backend	10–50 ms
Parsing	50–100 ms
LLM Analysis	1.7 s
Action Dispatch	50–100 ms
Logging	50 ms

$$T_{\text{end-to-end}} \approx 1.9\text{--}2.2\text{s}$$

Meets requirement: < 3 seconds end-to-end.

Storage Modeling

Assumptions:

- Avg alert size (JSON): 2 KB
- 240,000 alerts/day per node (upper bound)
- Compression ratio: 0.3 (using gzip)

$$\text{Daily Storage} = 2\text{KB} \times 240,000 \times 0.3$$

=144 MB/day≈4.3 GB/month

Comfortably supported by 2 TB SSD storage.

API Cost Modeling (GPT-4)

- **Alert** **Volume**
A projected load of **30,000 alerts/day** →
 $30,000 \times 30 = 900,000$ alerts/month.
This value is consistent with alert rates observed in large-scale security monitoring environments reported in NIST SP 800-94 [45].
- **Average Token Usage Per Alert**
- **Input tokens (~300):** Typical Falco/Syscall alert metadata (container info, process tree, file path, K8s metadata).
- **Output tokens (~200):** Severity score, threat classification, summary, and automated action list.
- Total = **500 tokens/alert**, aligning with developer-reported prompt sizes for security-analysis tasks [46], [47].
- **Model** **Pricing**
OpenAI's published pricing lists **GPT-4 Mini** at approximately **\$5 per 1M tokens** (input + output combined) [48].

4.5 Hardware Design Approach (Circuit Design, Components, PCB Layout)

This section describes the physical architecture, hardware components, network layout, and edge deployment scenario.

Edge Hardware Overview

The system runs on edge compute nodes supporting low-latency Falco detection and Kubernetes microservices.

Table 7: Selected Edge Node Specifications

Component	Specification
CPU	Intel Xeon E-2288G (8C/16T)
RAM	64 GB DDR4 ECC
Storage	2 × 2 TB NVMe SSD (RAID-1)
Network	Dual 10 GbE NICs
Optional AI Acceleration	NVIDIA T4 GPU
Power	UPS-backed (10 min hold-up)

Rationale:

- Provides sufficient CPU for Falco syscall capture.
- RAID-1 ensures high resilience.
- GPU optional for local LLM inference (future upgrade).

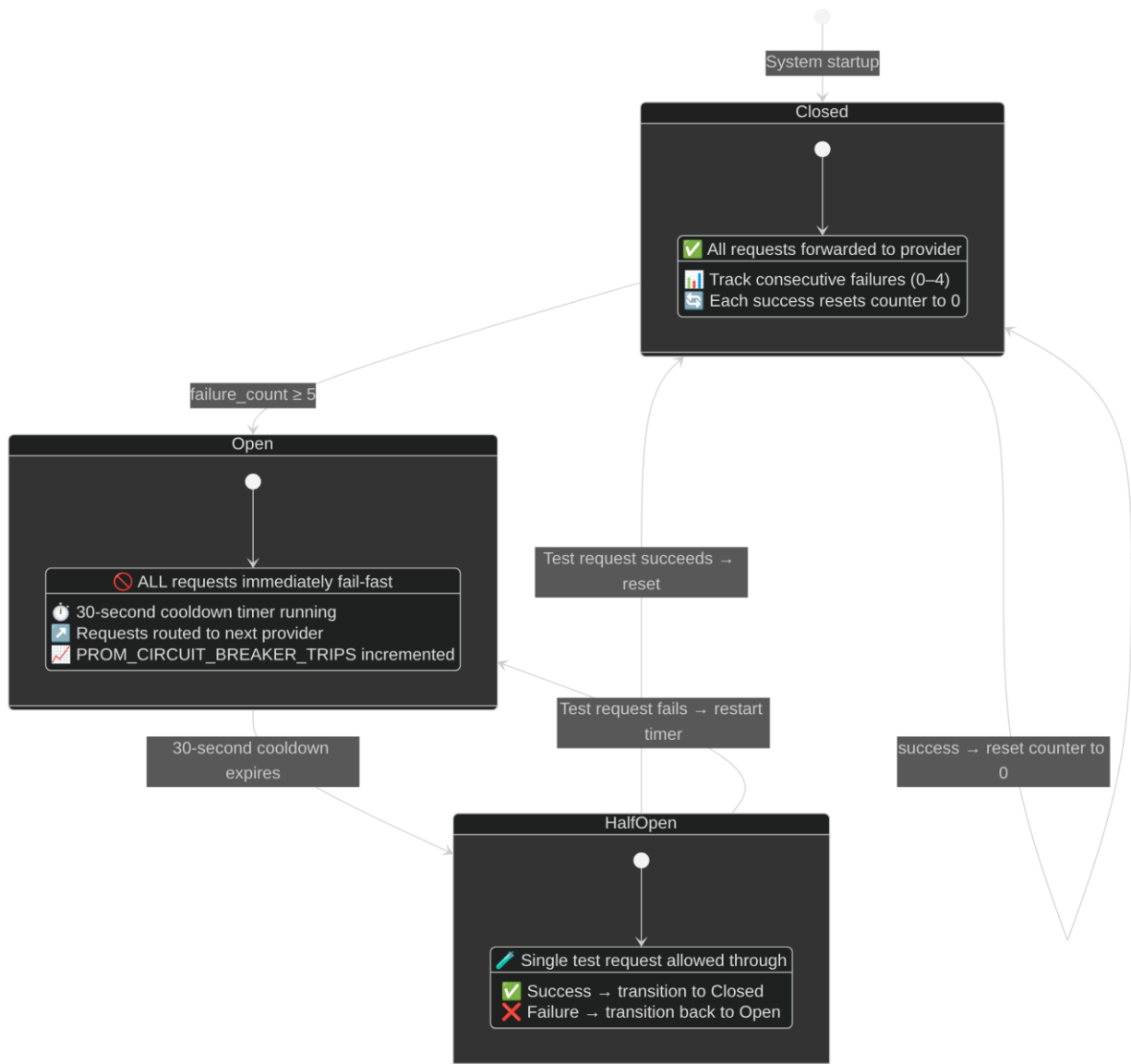


Figure 4: Circuit breaker state machine showing CLOSED, OPEN, and HALF_OPEN transitions

Hardware Security Considerations:

- NIC isolation prevents attackers from injecting packets into IDS sensors.
- UPS reduces downtime due to local power failure.
- Encrypted SSD ensures log confidentiality.
- TPM-enabled BIOS protects against physical tampering.

4.6 Software Design Approach (if applicable) (programming languages, algorithms, flowcharts).

This section describes the software architecture, algorithms, modules, APIs, and processing flow.

Technology Stack:

Backend

- Language: Python 3.11
- Framework: FastAPI
- Support libraries: Pydantic, SQLAlchemy, LangChain
- Security: OAuth2 + JWT

Frontend

- React + TypeScript
- Material-UI Components
- WebSocket for live alert feed

Orchestration

- Kubernetes 1.27
- Helm/Kustomize manifests
- Pod Security Standards

IDS

- Falco 0.36
- Custom ruleset for smart city behaviors

Data Processing Algorithm (Falco → LLM → Action)

```

Input: Raw Falco JSON Alert
Output: Enriched Alert + Recommended Action

1. Receive alert in FastAPI webhook
2. Extract process, container, and syscall metadata
3. Apply normalization (flatten JSON, remove PII)
4. Construct LLM prompt template
5. Send prompt via LangChain → GPT-4
6. Receive structured JSON:
{
  "summary": "...",
  "severity": "High",
  "threat_type": "Privilege Escalation",
  "mitigation": ["Isolate pod", "Block IP"],
  "confidence": 0.92
}
7. Validate JSON schema (fallback if invalid)
8. Forward "mitigation" to Kubernetes Operator
9. Log enriched alert to dashboard

```

Figure 5: LLM Processing Pipeline for Transforming Raw Falco Alerts into Structured Security Intelligence

LLM Prompt Template

You are an AI Security Analyst.

Analyze the Falco alert below and produce:

- Summary (≤ 50 words)

- Threat type
- Severity (Critical/High/Medium/Low)
- Recommended mitigation steps
- Confidence score (0 to 1)

Return valid JSON.

ALERT:

{{falco_event_json}}

Flowchart (Backend Processing)

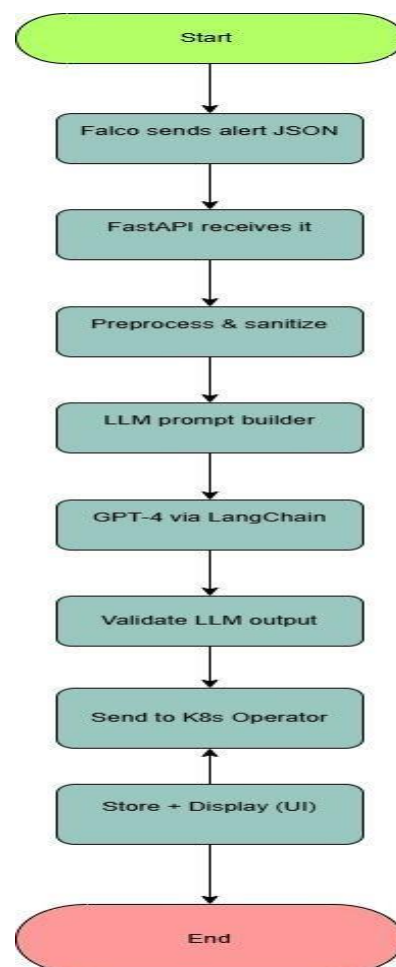


Figure 6: Flowchart (Backend Processing)

4.7 List of Required Components (Hardware & Software)

Hardware Components

Table 8: Hardware Components

Component	Quantity	Unit Cost	Total Cost
Dell R340 Edge Server	2	\$8,000	\$16,000
Cisco SG350 Switch	1	\$1,500	\$1,500
FortiGate Firewall	1	\$2,000	\$2,000
APC UPS 3000VA	2	\$1,500	\$3,000
SSD Storage Array	1	\$3,000	\$3,000
Total Hardware Cost			\$25,500

Software Components

Table 9: Software Components

Component	License	Cost
-----------	---------	------

Kubernetes	Open Source	\$0
Falco IDS	Open Source	\$0
PostgreSQL	Open Source	\$0
FastAPI	Open Source	\$0
LangChain	Open Source	\$0
GPT-4 API	Subscription	\$6,000/year
Monitoring Tools	Mixed	\$2,000/year

Total Annual Software Cost = \$8,000

4.8 Project Planning (Work Packages, Timeline & Milestones)

Work Breakdown Structure (WBS)

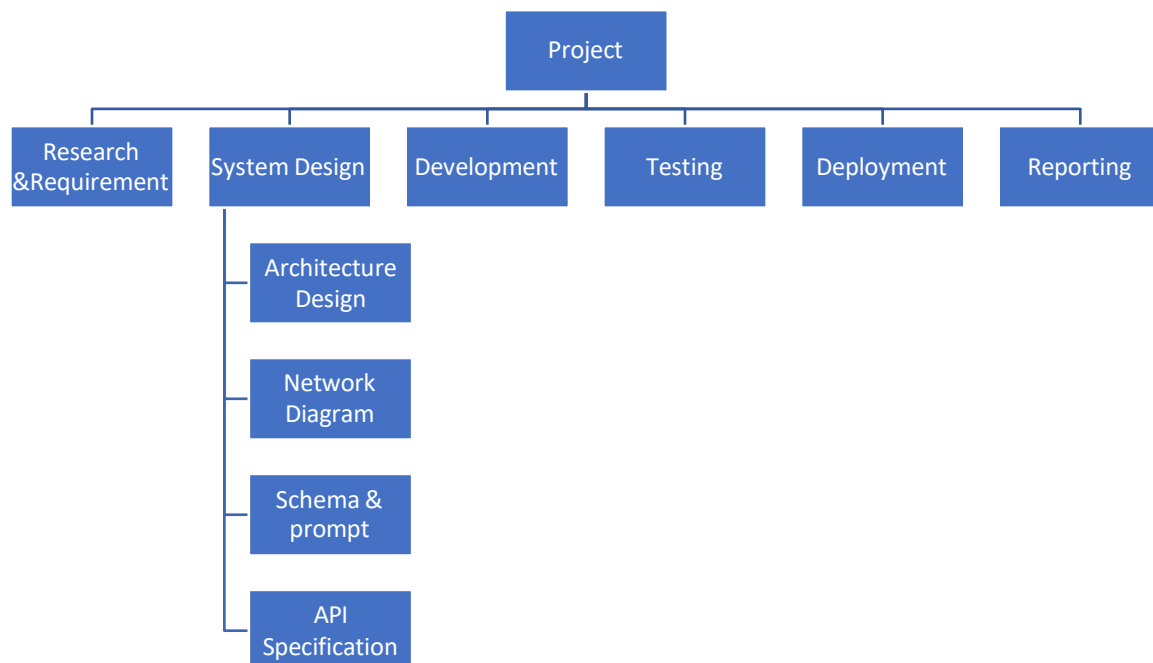


Figure 7: Work Breakdown Structure (WBS)

Gantt Chart (Capstone I)

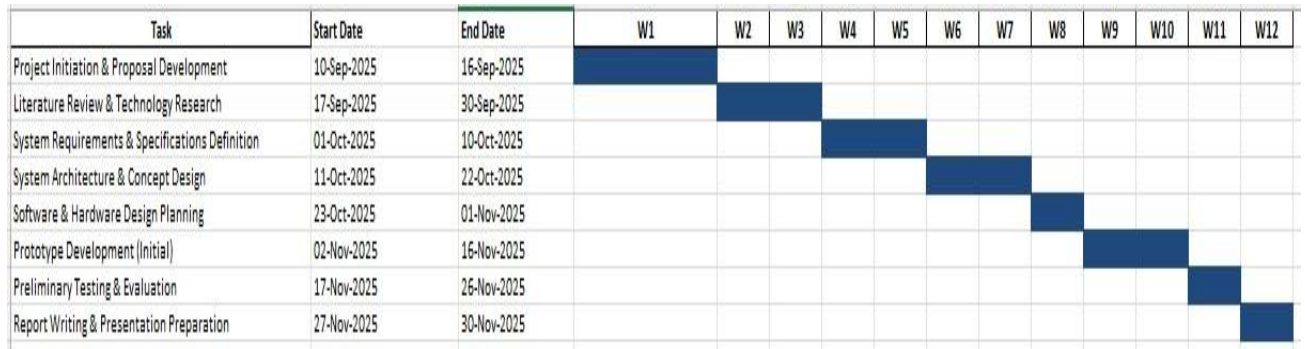


Figure 8: Gantt Chart (Capstone I)



Figure 9: Gantt Chart (Capstone II)

4.9 Project Budget

Table 10: Capital Expenditure (CAPEX)

Category	Cost
Hardware	\$25,500
Setup & Installation	\$5,000

Miscellaneous	\$4,500
Total CAPEX	\$35,000

Table 11: Operational Expenditure (OPEX)

Category	Annual Cost
GPT-4 API	\$6,000
Monitoring tools	\$2,000
Cloud services	\$12,000
Maintenance	\$5,000
Total OPEX	\$25,000/year

Table 12: Development Costs

Category	Cost
Personnel (3 × 9 months)	\$225,000
Training	\$10,000

Testing	\$15,000
---------	----------

Total Development Cost = \$250,000

4.10 Project Management (Team Structure, Roles, and Responsibilities)

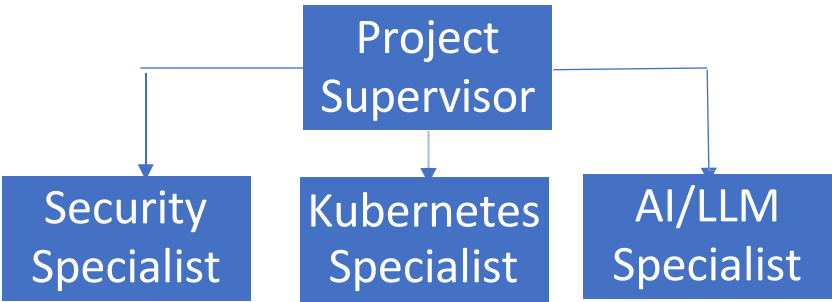


Figure 10: Project Management (Team Structure)

Table 13: Roles

Role	Name	Responsibilities
------	------	------------------

Kubernetes Specialist	Abdallahi Mahmoud	K3 Cluster Setup - Automated Response Time- System Integration -
LLM Specialist	Ali Suhail	LLM Integration - Threat Analysis - Plain English Explanations - Action Recommendations -
Security Specialist	Khaled Rahman	Deploying Falco IDS - Creating attack scenarios - Validating detection -

4.11 Expected Deliverables: (What the project will produce at the end of Capstone 1 and Capstone 2)

The project will produce several key deliverables across Capstone I and Capstone II to ensure structured progress, documentation, and validation of the proposed LLM-driven Intrusion Detection System. These deliverables reflect both the research and implementation components of the system.

Capstone I Deliverables:

These deliverables focus on research, system design, preliminary modeling, and early-stage prototyping.

1. Project Proposal -

A complete written proposal outlining the problem, objectives, motivation, and expected impact.

2. Draft and Final Capstone I Report -

Full documentation including introduction, literature review, requirements, design methodology, modeling, and preliminary results.

3. System Architecture Diagram –

High-level diagram showing Falco → Backend → LLM → Kubernetes Flow.

4. Concept Generation & Morphological Matrix –

Evaluation of alternative system designs.

5. Modeling & Calculations -

Latency model, LLM token usage estimation, and scalability modeling.

6. Prototype Demonstration (Basic Version) –

Early prototype showing:

- Falco generating alerts
- Backend receiving and parsing alerts
- LLM returning summaries and severity

7. Gantt Chart & Work Breakdown Structure (WBS) –

Project planning documents for Capstone I & II.

8. Capstone I Presentation -

Summarizing project design, results, and plans for Capstone II.

Capstone II Deliverables:

These deliverables focus on system implementation, testing, evaluation, and validation.

1. Fully Functional Prototype -

End-to-end LLM-based IDS integrated with Falco and Kubernetes automation.

2. API Integration Module -

Backend interface capable of reliable, structured communication with GPT-4 or fallback LLMs.

3. Kubernetes Operator for Automated Responses -

Module that applies automated or semi-automated mitigation actions.

4. Performance Evaluation Report - Includes:

- Latency measurements
- Accuracy & consistency tests
- Stress and scalability testing
- Comparison with baseline IDS performance

5. User Interface or Dashboard (Optional) -

For viewing alerts, LLM summaries, and system behavior.

6. Testing & Validation Results -

Validated using simulated attacks and real Falco logs.

7. Final Capstone II Report -

Complete system documentation with implementation, evaluation, discussion, and conclusion.

8. Final Capstone II Presentation & Demonstration -

Live demonstration of the working system in front of supervisor.

5 Chapter 5: Implementation

This chapter outlines the practical execution of the Capstone II system. It involves the design and integration of the constituent hardware, software, and prototyping environment into a working LLM-based intrusion detection system. Details on system construction, iterative design improvements, and cost benefit assessment of the proposed system are also included in this chapter.

.

5.1 Hardware Implementation:

The goal of Capstone II's hardware implementation is to incorporate a tangible smart environment element into the LLM-driven IDS architecture. In a small-scale smart city setting, a Raspberry Pi 5 served as a representative edge node. An AM312 PIR motion sensor was attached to the Raspberry Pi in order to replicate the detection of physical events in the real world.

GPIO pins (VCC, GND, and signal output) were used to connect the PIR sensor to the Raspberry Pi. The Raspberry Pi functions as a thin and light edge device that can create and send structured events to the IDS pipeline. To guarantee consistent communication with the Kubernetes-based monitoring environment, Ethernet connectivity was utilised.

In order to demonstrate detection, analysis, and response from both digital and physical sources, this hardware configuration serves as a simplified smart infrastructure node.

.

5.2 Software Development

Capstone II software development entailed improving and expanding the detection-decision-response pipeline from Capstone I. Among the crucial development activities were:

- Multi-Provider LLM support implementation (OpenAI fallback, Groq primary)
- Enforcing JSON outputs through structured prompt engineering
- Including failover logic
- Putting governance execution modes into practice
- Mechanisms for rate limitation and alert deduplication
- Integration of remediation APIs for Kubernetes
- Layer of database logging for persistence

Python and FastAPI are used in the implementation of the backend system. NetworkPolicy isolation is one example of an automated remediation action that uses Kubernetes APIs. Prior to execution, consistent LLM output is ensured by structured JSON validation.

This phase involved response-time optimisation, modular refactoring, and code debugging.

5.3 Prototype Development:

A comprehensive small-scale smart security environment is represented by the Capstone II prototype. The prototype consists of:

- Smart service simulations (parking system, healthcare API, camera)
- Intrusion detection at runtime for Falco
- Suricata network tracking (validation of integration)
- Contextual threat analysis using LLM
- Remedial control based on governance

- Mitigation measures using Kubernetes
- Physical sensing using a Raspberry Pi

End-to-end functionality is demonstrated by the prototype:

1. Finding suspicious activity
2. Organized threat analysis of LLM
3. Decision-making based on governance
4. Automated implementation of remediation
5. Persisted logging

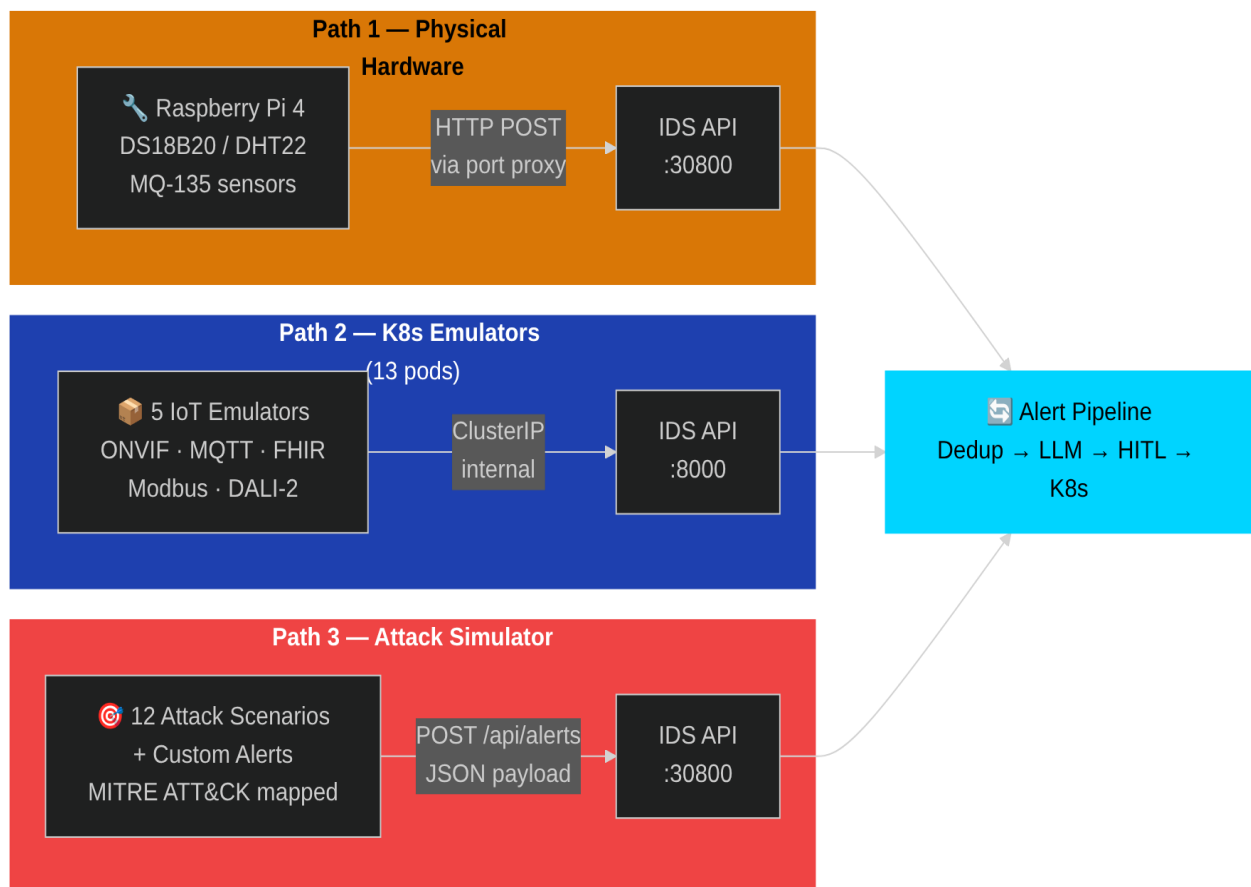


Figure 11: IoT integration architecture showing device types, protocols, and data flow into the IDS.

5.4 Economic Evaluation:

The economic feasibility of the project was evaluated using a simplified Business Model Canvas (BMC) framework.

Key components include:

- **Key Partners:** We rely on cloud service providers, LLM vendors such as Groq or OpenAI, cybersecurity tool providers, and Kubernetes infrastructure platforms to support the overall system operation.
- **Key Activities:** The main activities include continuously monitoring security alerts, analyzing threats using LLMs, executing appropriate remediation actions, and maintaining system performance and reliability.
- **Key Resources:** Our essential resources include access to LLM APIs, the Kubernetes cluster environment, IDS tools like Falco and Suricata, the monitoring stack, and the edge hardware used for deployment.
- **Value Propositions:** The project delivers an intelligent intrusion detection system that combines explainable AI analysis with controlled and safe automation for smart city edge environments.
- **Customer Relationships:** The system is designed to support long-term operational use by providing monitoring visibility, decision transparency, and flexible automation control for operators.
- **Customer Segments:** The primary users include smart city operators, municipal IT departments, and organizations managing edge computing infrastructure.
- **Channels:** The solution can be deployed directly within existing smart city infrastructure and integrated with monitoring dashboards used by security

teams.

- **Cost Structure:** Major costs include cloud hosting services, LLM API usage, hardware components, and ongoing system maintenance.
- **Revenue Streams:** The solution could potentially be offered as a subscription-based security monitoring service with AI-assisted threat analysis and automated response capabilities.

6 Testing and Results

The testing plan and assessment findings for the deployed system are presented in this chapter. It describes how simulated attack scenarios, unit testing, and integration testing were used to validate the system. The chapter assesses whether the implementation satisfies the specified requirements, examines observed limitations, and analyses system performance.

6.1 Testing Methodology:

A multi-level approach was used for testing:

- Ingestion, prompt construction, and governance logic unit tests
- Testing the detection-decision-response pipeline's integration
- Validation of failover between LLM providers
- To verify remediation execution, attack scenarios were simulated

Both functional correctness and system reliability are validated by the testing methodology.

6.2 Experimental Setup:

Included in the experimental setup are:

- K3s Kubernetes cluster with a single node
- Integrated Suricata for network detection
- Groq as the main supplier of LLM

- OpenAI as a backup source
- Using a PIR sensor as edge hardware, the Raspberry Pi 5
- Backend for alert ingestion using FastAPI

Outbound connection anomalies, suspicious file access, and simulated privilege escalation attempts were among the testing conditions.

6.3 Results:

Preliminary testing validates:

- Successfully ingesting runtime detection alerts
- LLM-generated structured JSON output
- Appropriate handling of provider failures
- The ability to switch between governance modes
- Automated Kubernetes isolation policy execution
- Effective identification of PIR-triggered events

In Draft 2, a thorough statistical benchmarking will be provided.

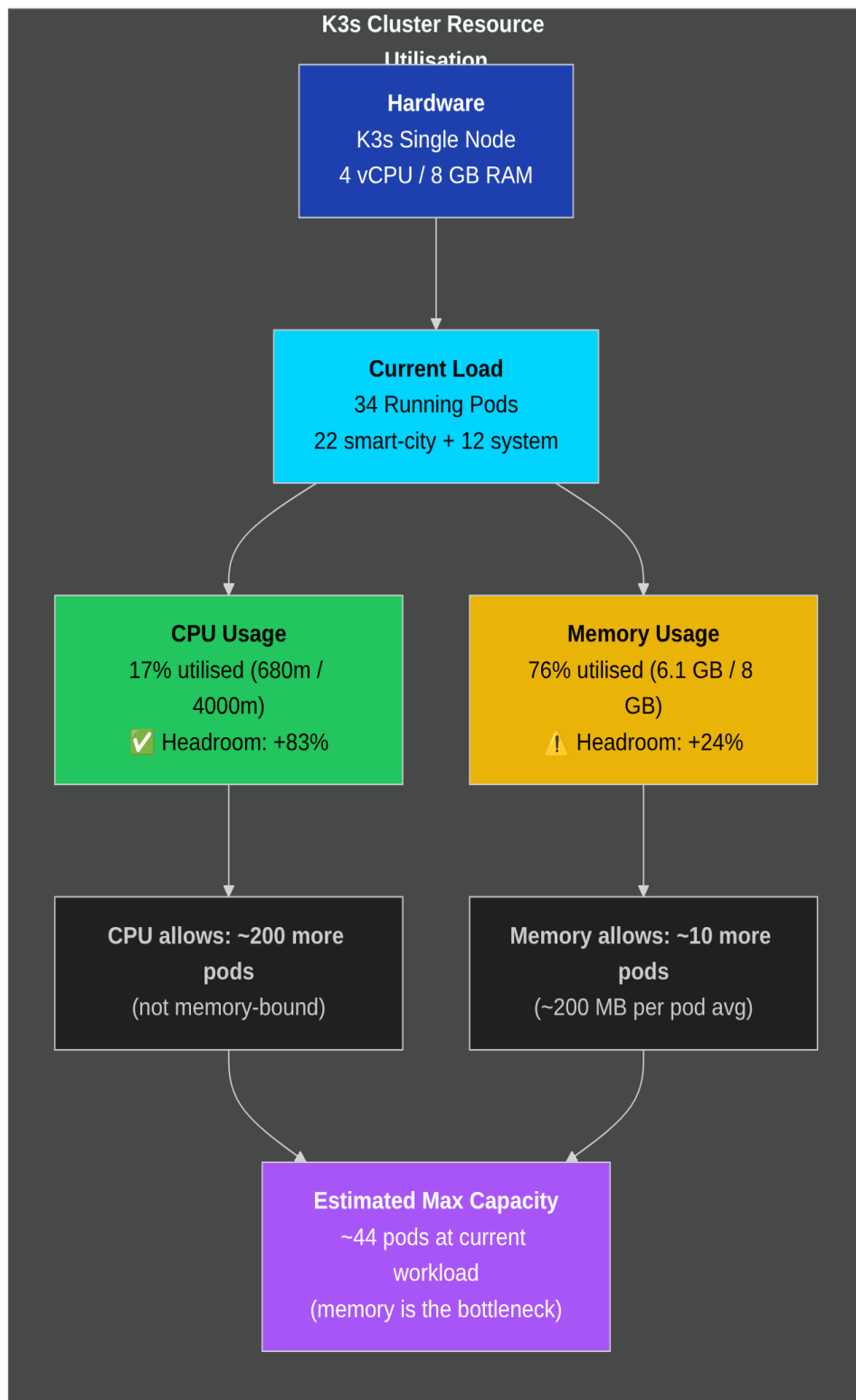


Figure 12: Cluster scalability and resource utilization under increasing alert load.

6.4 Performance Evaluation:

Under moderate alert volume, preliminary latency observations show a stable system response. Deduplication and rate-limiting techniques decreased redundant LLM calls and increased processing stability.

By lowering reliance on a single API provider, the multi-provider architecture improves availability. In the upcoming draft version, quantitative benchmarking under stress testing will be extended.

6.5 Limitations and Challenges:

A number of restrictions were noted:

- Variability in external LLM API latency
- Limiting the API rate when there are a lot of alerts
- The default IDS rule sets' alert noise
- Reliance on third-party cloud connectivity
- Network-level detection partially validated

These restrictions point to areas that should be optimized in subsequent stages.

6.6 Improvements and Optimization:

Here's what we're planning to do to make things better:

- Run a bunch of tests to see how the system holds up
- Make prompts use fewer tokens
- Index the database so it saves data more quickly
- Make the dashboard look better
- Check out using Kubernetes across different machines
- Tweak the governance settings some more

7 Conclusion and Future Work

Okay, so this chapter gives you the rundown on how Capstone II went. We'll talk about what we got done with the system – getting it built, tested, and making sure it worked right. Plus, we'll look at what the project added and chat about ways to make it even better down the road.

7.1 Project Summary:

Capstone II builds on the Capstone I setup, making it more reliable. Now, it uses several LLMs from different sources for backup, has automation controlled by governance, cuts down on alert spam, and simulates real-world smart environments with Raspberry Pi stuff.

We've checked out the detect, decide, and respond process with a prototype.

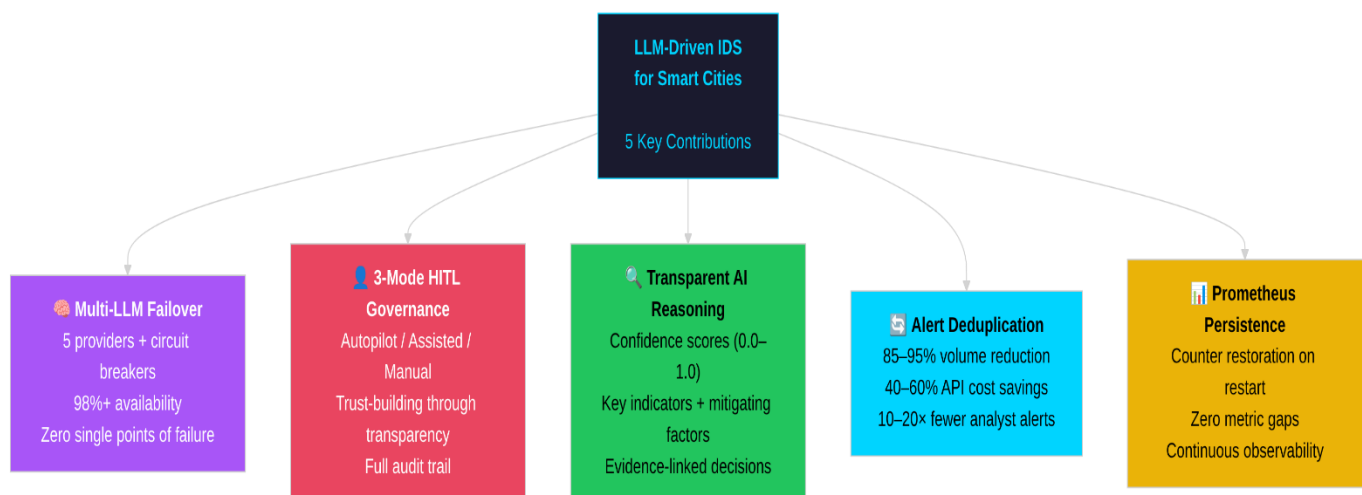


Figure 13: Key contributions map showing the five novel contributions of this capstone project.

7.2 Contributions and Achievements:

Major contributions include:

- Multi-provider LLM integration with failover
- Governance-controlled remediation modes
- Explainable AI output with confidence scoring
- Alert deduplication and rate limiting
- Raspberry Pi smart environment integration
- Persistence for auditability
- Modular and scalable architecture design

These contributions align with modern intelligent edge-security research trends.

7.3 Future Enhancements:

Future enhancements include:

- Full statistical performance benchmarking
- Large-scale stress testing
- Multi-node deployment validation
- Dashboard and UI enhancement
- Extended attack scenario library
- Optimization of database performance

8 References

- [1] A. Zanella et al., "Internet of things for smart cities," *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22-32, 2014.
- [2] M. Mohammadi et al., "Deep learning for IoT big data and streaming analytics: A survey," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, pp. 2923-2960, 2018.
- [3] P. Bellavista et al., "A survey on fog computing for the Internet of Things," *Pervasive and Mobile Computing*, vol. 52, pp. 71-99, 2019.
- [4] Sysdig, "Falco: Cloud-native runtime security," 2023. [Online]. Available: <https://falco.org/>
- [5] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.
- [6] A. Q. Jiang et al., "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.
- [7] J. Jin et al., "An information framework for creating a smart city through Internet of Things," *IEEE Internet of Things Journal*, vol. 1, no. 2, pp. 112-121, 2014.
- [8] H. Andrade, M. F. Balbi, and E. Monteiro, "Cybersecurity Challenges in Smart Cities: A Systematic Review," *IEEE Access*, vol. 11, pp. 77482–77510, 2023.
- [9] M. Naphade et al., "Smarter cities and their innovation challenges," *Computer*, vol. 44, no. 6, pp. 32-39, 2011.
- [10] M. Roesch, "Snort - lightweight intrusion detection for networks," *LISA '99: Proceedings of the 13th USENIX Conference on System Administration*, 1999.

- [11] V. Julisch et al., "Open Information Security Foundation: Suricata IDS/IPS," 2023.
- [12] A. Khraisat et al., "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 20, 2019.
- [13] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," *2010 IEEE Symposium on Security and Privacy*, 2010.
- [14] B. Schneier, "Attack trees," *Dr. Dobbs's Journal*, vol. 24, no. 12, pp. 21-29, 1999.
- [15] T. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877-1901, 2020.
- [16] H. Touvron et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [17] H. Pearce et al., "Examining zero-shot vulnerability repair with large language models," *2023 IEEE Symposium on Security and Privacy*, 2023.
- [18] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," *2010 IEEE Symposium on Security and Privacy*, Oakland, CA, 2010, pp. 305-316.
- [19] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153-1176, 2016.
- [20] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems," *2015 MilCIS*, Canberra, 2015.

- [21] R. Vinayakumar et al., "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525-41550, 2019.
- [22] G. Apruzzese et al., "On the effectiveness of machine and deep learning for cyber security," *2018 CyCon*, Tallinn, 2018.
- [23] A. Q. Jiang et al., "Mistral 7B," *arXiv:2310.06825*, 2023.
- [24] Microsoft Security, "Using GPT-4 for Security Incident Analysis," *Microsoft Security Blog*, 2023.
- [25] D. Kang et al., "Exploiting programmatic behavior of LLMs for penetration testing," *arXiv:2302.05733*, 2023.
- [26] W. Shi et al., "Edge computing: Vision and challenges," *IEEE IoT Journal*, vol. 3, no. 5, pp. 637-646, 2016.
- [27] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30-39, 2017.
- [28] L. De Donno et al., "Foundations and evolution of modern computing paradigms," *IEEE Access*, vol. 7, pp. 150936-150948, 2019.
- [29] D. Bernstein, "Containers and cloud: From LXC to Docker to Kubernetes," *IEEE Cloud Computing*, vol. 1, no. 3, pp. 81-84, 2014.
- [30] T. Chen et al., "Securing container-based systems: Challenges and solutions," *ACM Computing Surveys*, vol. 54, no. 7, 2021.

- [31] S. Ramirez, "FastAPI: Modern, fast (high-performance), web framework for building APIs with Python," FastAPI Documentation, 2023. [Online]. Available: <https://fastapi.tiangolo.com/>
- [32] M. Hausenblas and S. Schimanski, "Programming Kubernetes: Developing Cloud-Native Applications," O'Reilly Media, 2019.
- [33] B. Burns et al., "Kubernetes: Up and Running: Dive into the Future of Infrastructure," 2nd ed., O'Reilly Media, 2019.
- [34] L. Lukša, "Kubernetes in Action," 2nd ed., Manning Publications, 2020.
- [35] K. T. Ulrich and S. D. Eppinger, "Product Design and Development," 6th ed., McGraw-Hill Education, 2015.
- [36] D. G. Ullman, "The Mechanical Design Process," 5th ed., McGraw-Hill Education, 2015.
- [37] T. Ritchey, "Wicked Problems – Social Messes: Decision Support Modelling with Morphological Analysis," Springer, 2011.
- [38] IEEE, "IEEE Standard for Software and System Test Documentation," IEEE Std 829-2008, 2008.
- [39] S. McConnell, "Code Complete: A Practical Handbook of Software Construction," 2nd ed., Microsoft Press, 2004.
- [40] D. Votipka et al., "Understanding security mistakes developers make: Qualitative analysis from Build It, Break It, Fix It," *USENIX Security Symposium*, 2020, pp. 109-126.
- [41] M. Silic and A. Back, "Information security: Critical review and future directions for research," *Information Management & Computer Security*, vol. 22, no. 3, pp. 279-308, 2014.

- [42] P. Bhatt et al., "The operational role of security information and event management systems," *IEEE Security & Privacy*, vol. 12, no. 5, pp. 35-41, 2014.
- [43] N. Papernot and P. McDaniel, "Deep k-nearest neighbors: Towards confident, interpretable and robust deep learning," *arXiv preprint arXiv:1803.04765*, 2018.
- [44] S. Russell, "Human Compatible: Artificial Intelligence and the Problem of Control," Viking Press, 2019.
- [45] **K. Scarfone and P. Mell**, "Guide to Intrusion Detection and Prevention Systems (IDPS)," *NIST Special Publication 800-94*, Feb. 2007.
- [46] **OpenAI Developer Community**, "Typical token usage patterns for system prompts and analysis prompts," forum discussions, 2023–2024. Available: <https://community.openai.com>
- [47] **Signoz**, "Benchmarking LLM latency: real-world measurements for GPT-3.5 and GPT-4," *Signoz Engineering Blog*, 2023. Available: <https://signoz.io>
- [48] **OpenAI**, "Pricing for GPT-4, GPT-4 Turbo, and GPT-4 Mini," *OpenAI Pricing Documentation*, 2024. Available: <https://platform.openai.com/pricing>

9 Appendices

Appendix A – Falco Ruleset (Custom Rules)

Appendix B – Sample Falco JSON Alerts

Appendix C – Backend API (FastAPI) Code Snippets

Appendix D – LLM Prompt Templates and Output Schema

Appendix E – System Architecture Diagrams (High-Resolution)

Appendix F – Work Breakdown Structure (WBS) Full Diagram

Appendix G – Gantt Chart (Capstone I) – Full Page

Appendix H – Hardware Specifications and Benchmarks

Appendix I – Kubernetes Deployment Manifests (YAML Files)

Appendix J – Test Cases and Validation Screenshots

Appendix K – Simulation Logs and Performance Metrics

Appendix L – Ethical Approval / Data Handling Compliance Forms

Appendix M – Additional Tables and Calculations

Appendix N – Risk Register (Full Version)